

Slovenská technická univerzita v Bratislave

Fakulta informatiky a informačných technológií

IA-TARA

Tímový projekt

Autori:

Bc. Marko Šebest

Bc. Samuel Švenk

Bc. Oliver Nemček

Bc. Richard Blažo

Bc. Danylo Tokarskyi

Bc. Martin Čulen

Obsah

1	Úvod	3
2	TARA	3
3	Databázy	5
3.1	MITRE ATT&CK	5
3.2	MITRE CAPEC	6
3.3	MITRE CWE	6
4	Návrh	7
4.1	Design	7
4.2	Definovanie systému	8
4.2.1	Komponenty	8
4.2.2	Dátové entity	8
4.2.3	Technológie	8
4.3	Identifikácia rizík	9
4.3.1	Asistent scenárov poškodenia	9
4.3.2	Asistent scenárov hrozieb	9
4.3.3	Asistent krokov útoku	9
4.3.4	Controls	9
4.4	Analýza rizík a mitigácia rizík	10
4.4.1	Control skupiny	10
4.4.2	Využitie LLM	10
4.5	Backend	11
5	Frontend	15
5.1	Návrh a prototyp aplikácie	15
6	Backend - druhá iterácia	16
6.1	Autentizácia a autorizácia	16
6.2	Správa projektov	17
6.3	Príklad použitia backendu	18
7	Výsledná implementácia	26
7.1	Beh a nasadenie aplikácie	26
7.2	Celkový tok práce v aplikácii	26
7.3	Autentizácia, projekty a prístupové práva	27
7.4	Dátový model výslednej aplikácie	27
7.4.1	Technológie	28
7.4.2	Komponenty	28
7.4.3	Data entities	28
7.4.4	Threat classes	28
7.4.5	Attack steps	28
7.4.6	Threat scenarios	29
7.4.7	Damage scenarios a concerns	29
7.4.8	Controls a control classes	30
7.4.9	Control groups	30
7.4.10	Risk treatment a cybersecurity goals	30
7.5	Výpočet attack feasibility, impact a risk level	31
7.6	Backend API a synchronizácia vzťahov	32
7.7	Grafický pohľad	32

7.8	Strom projektu a vytváranie súvisiacich objektov	33
7.9	Detailový panel a formuláre	33
7.10	Tabuľkový pohľad	34
7.11	Control skupiny a porovnávanie mitigácií	35
7.12	Asset Identification assistant	35
7.13	Konzola a LLM návrhy	36
7.14	MITRE integrácia	36
7.15	Generovanie PDF reportu	37
7.16	Vzťah medzi grafom, tabuľkami, detailmi a backendom	38
7.17	Príklad práce s aplikáciou	38
7.18	Obmedzenia a možné rozšírenia	39
7.19	Zhrnutie výsledku	39

1 Úvod

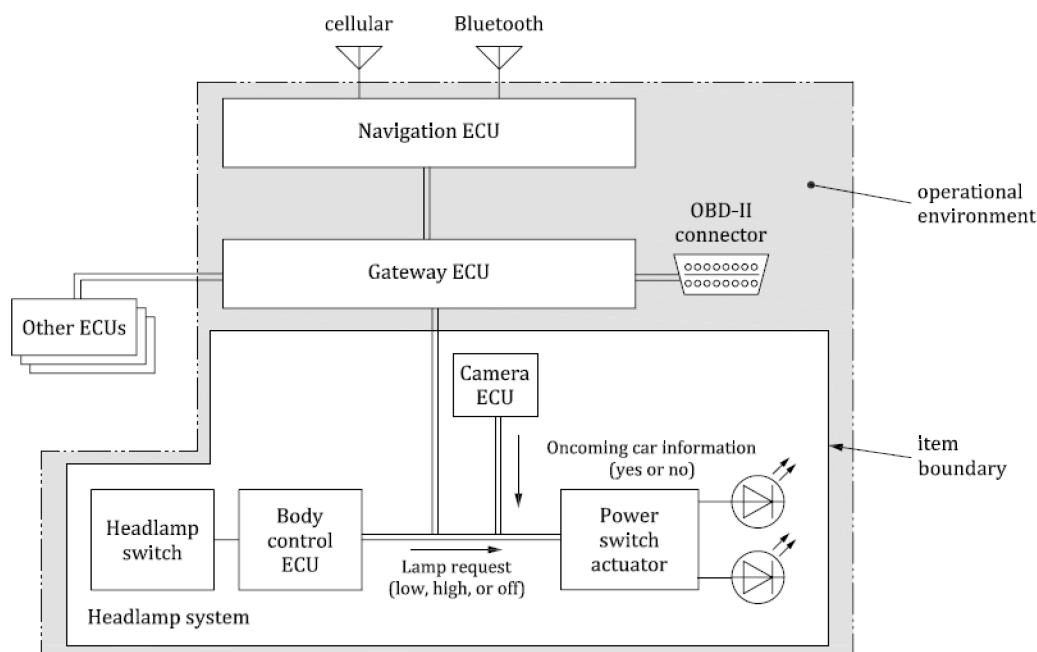
TARA (Threat Analysis and Risk Assessment) je systematický proces identifikácie, analýzy a hodnotenia hrozieb a rizík, ktoré môžu ovplyvniť bezpečnosť systému, produktu alebo služby. Cieľom analýzy je nájsť potenciálne zraniteľnosti, identifikovať útoky, ktoré by mohli tieto zraniteľnosti využiť, určiť výsledný dopad takýchto útokov a zistiť, ako je možné sa proti nim efektívne brániť.

TARA sa využíva najmä v oblastiach, kde je bezpečnosť kritická – ako napríklad v automobilovom priemysle alebo pri vývoji rôznych kritických embedded systémov. Jej výsledkom je prehľad rizík a odporúčaní, ktoré slúžia ako základ pre návrh vhodných bezpečnostných opatrení a zvyšujú celkovú odolnosť systému voči hrozbám.

2 TARA

TARA analýza pozostáva z nasledujúcich krokov:

1. Definícia „**Item**“-ov



Obr. 1: Príklad **Itemu** analyzovaného v rámci TARA

- **Item** = nejaký komponent systému, ktorý sa bude skúmať, na ktorom sa bude robiť TARA.
- Definícia jeho funkcií, aké má hranice, vzťah k ostatným častiam systému.
- TARA sa robí na každý item v systéme, ktorý je relevantný z hľadiska kybernetickej bezpečnosti.
- Definícia itemu nie je časť samotnej TARA, ale TARA sa vykonáva na definovaných itemoch.

2. Definícia „**Asset**“-ov

- **Asset** je niečo, čo má hodnotu, čo musí byť chránené.
- CIA

3. Určenie „**Damage scenarios**“

- Čo sa môže stať, ak by bol stratený asset? Aké sú **dôsledky** – čo zlé sa môže stať?
- Napríklad, ak by sa pokazila parkovacia kamera, tak damage scenario by mohlo byť, že vodič pri parkovaní narazí do auta za ním, alebo že musí použiť spätné zrkadlo.
- Každé damage scenario má priradený „impact“ – napr. to, že na cúvanie sa musí použiť spätné zrkadlo, má menší impact ako niečo, čo spôsobí čelnú kolíziu.
- Existujú štyri typy damage scenarios: „safety“, „financial“, „personal“, „operational“

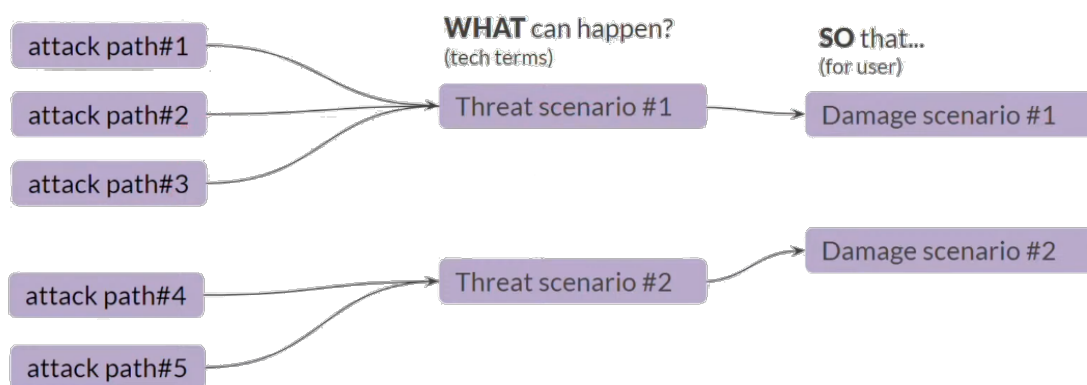
4. Určenie „**Threat scenarios**“

- Čo môže spôsobiť damage scenario?
- Napr. čo sa môže stať, aby to vyústilo v to, že sa pokazí parkovacia kamera – môže sa pokaziť senzor, niekto sa na kameru mohol napojiť, atď.
- Opísané na vysokej úrovni, bez bližších detailov.

5. Určenie „**Attack paths**“

- Ako zrealizovať threat scenario?
- Napr. ako by sa útočník vedel napojiť na kameru – detailný opis krok za krokom.
- Pre jedno threat scenario môže existovať viac attack paths.
- Každý attack path má **feasibility rating** – ako ťažké je vykonať daný útok (napr. ak na vykonanie útoku musí útočník byť fyzicky v aute, znižuje to jeho feasibility rating).

HOW can this happen?



Obr. 2: Vzťahy medzi **attack paths**, **threat scenarios** a **damage scenarios**

6. Kombináciou **impact ratingu** a korešpondujúceho **feasibility ratingu** sa vypočíta **risk value** – číslo medzi 1 a 5.

		Attack feasibility rating			
		Very Low	Low	Medium	High
Impact rating	Severe	2	3	4	5
	Major	1	2	3	4
	Moderate	1	2	2	3
	Negligible	1	1	1	1

Obr. 3: Príklad tabuľky pre určenie **risk value**

7. Vytvorenie „**risk treatment options**“ – ako sa dá risk eliminovať (cybersecurity goals), alebo rozhodnutie, že je risk akceptovateľný a mal by sa ponechať (cybersecurity claims).

Kroky 3, 4, 5 sa môžu robiť v ľubovoľnom poradí, t. j. môžeme začať otázkou „Čo sa môže stať?“ (threat scenario), z toho určiť, aké to má následky (damage scenarios), a ako sa to môže stať (attack paths). V praxi sa to nerobí krok za krokom, ale jednotlivé kroky TARA analýzy sa vykonávajú v poradí, ktoré je pre daný prípad najvhodnejšie. Poradie samotné teda nie je dôležité, záleží iba na vzťahoch medzi jednotlivými threat scenarios, damage scenarios a attack paths navzájom.

3 Databázy

3.1 MITRE ATT&CK

MITRE ATT&CK (**Adversarial Tactics, Techniques, and Common Knowledge**) je databáza, ktorá katalogizuje reálne pozorované taktiky, techniky a postupy (TTP) používané útočníkmi pri kybernetických útokoch. Je postavená na reálnych dátach zo zaznamenaných bezpečnostných incidentov a slúži na pochopenie toho, ako útočníci v praxi postupujú v jednotlivých fázach útoku.

ATT&CK je organizovaný do viacerých **matic** (napr. Enterprise, Mobile, ICS), ktoré popisujú správanie útočníkov v rôznych prostrediach.

ATT&CK Enterprise sa zameriava na tradičné business IT prostredie – počítače, servery, cloud, identity, siete. Obsahuje techniky, ktoré útočníci používajú pri útoku na firemné informačné systémy (phishing, privilege escalation, exfiltrácia dát, atď.).

ATT&CK Mobile sa zameriava na útoky na mobilné zariadenia (Android, iOS). Matica zahŕňa techniky neautorizovaného prístupu k zariadeniam, vrátane tých, ktoré nevyžadujú fyzický prístup k zariadeniu.

ATT&CK ICS (Industrial Control Systems) je zameraný na riadenie priemyselných procesov – bezpečnostné kontroléry, priemyselné protokoly, atď. Techniky obsahujú napríklad útoky na fyzické procesy, manipuláciu signálov, narušenie prevádzky alebo bezpečnostných mechanizmov. Je veľmi blízka oblasti embedded systémov a kritickej infraštruktúry.

- Mapuje techniky (typy útokov) na taktiky (ciele útokov). Technika môže patriť do niekoľkých taktík.
- Dostupné verzie pre enterprise systémy, mobily a ICS. Pre autá je najviac použiteľná mobilná matica, prípadne matica ICS.

- Obranné opatrenia proti technikám: mitigations, detections, assets (nie je dostupné pre mobility).

Databáza ATT&CK je dostupná vo formáte STIX dokumentu (podobný ako JSON). Jej najaktuálnejšiu podobu je možné získať aj pomocou API.

Myslíme, že pre náš projekt by bolo vhodné načítať a analyzovať tento STIX súbor a následne na jeho základe pripraviť dáta vo vlastnom formáte, ktorý je vhodnejší pre náš konkrétny prístup.

Vytvorili sme experimentálny TS skript, ktorý vie tento dokument načítať a spracovať. Na jeho základe sme zistili, že typy objektov, ktoré dokument obsahuje a ktoré sú použiteľné pre náš projekt, sú „**attack-pattern**“, „**course-of-action**“, „**x-mitre-detection-strategy**“, „**tool**“ (prípadne) a „**relationship**“.

Okrem toho sme na základe JSON schémy STIX formátu vytvorili sériu TypeScriptových typov, ktoré umožnia jednoduchšie využívanie a spracovanie takýchto dát.

3.2 MITRE CAPEC

MITRE **CAPEC (Common Attack Pattern Enumeration and Classification)** je verejne dostupná databáza známych **attack patterns** (vzorov útokov), ktorá slúži ako dokumentácia toho, ako útočníci typicky postupujú pri zneužívaní zraniteľností. CAPEC klasifikuje a popisuje jednotlivé útoky formou „**attack patterns**“ – každý obsahuje informácie o mechanizme útoku, predpokladoch, možných dopadoch, potenciálnych mitigáciách a súvisiacich zraniteľnostiach.

Hoci CAPEC aj ATT&CK sú oba projekty spravované MITRE, oba dokumentujú útoky na rôznych úrovniach detailu:

CAPEC opisuje vysoko abstraktné vzory útokov, teda čo útočník robí, aký je samotný *princíp* útoku.

ATT&CK popisuje konkrétne taktiky, techniky a postupy (TTP) reálnych útočníkov v rôznych fázach útoku, teda ako reálne tieto útoky prebiehajú v praxi.

Medzi oboma systémami však existujú prepojenia – mnohé ATT&CK techniky odkazujú na príslušné CAPEC vzory, na ktorých sú založené. CAPEC je teda vhodný na pochopenie širšej logiky a princípov útokov, zatiaľ čo ATT&CK je orientovaný na konkrétne scenáre používané v reálnom svete.

3.3 MITRE CWE

CWE sa zaoberá najmä implementačnými/návrhovými slabínami softvéru, ktoré je potom možné využiť na napadnutie softvéru.

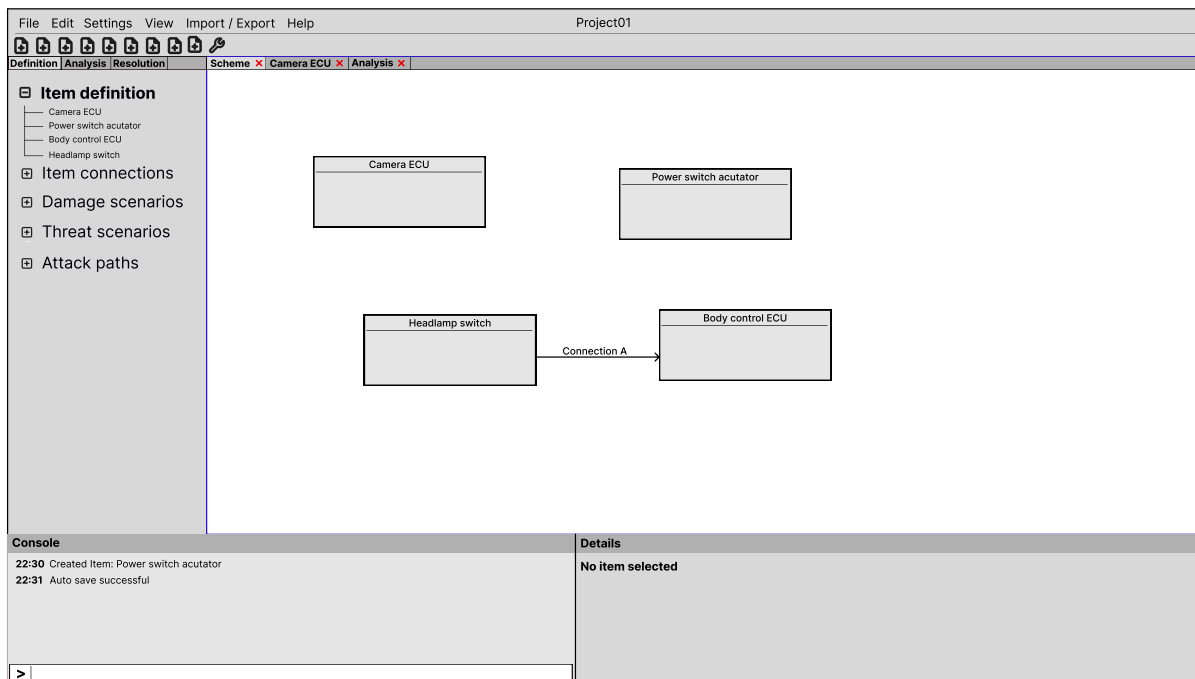
Databáza CWE má tvar hierarchického stromu, ktorého prvkami sú uzly. Každý uzol definuje nejakú možnú zraniteľnosť systému a obsahuje meno, popis, časť životného cyklu systému, v ktorom sa objaví, potenciálne riešenia, výsledky, platné platformy a odkazy na vzory útokov z CAPEC, ktoré používajú túto slabinu.

V závislosti od hĺbky môže mať každý uzol typ „Pillar“, „Class“, „Base“ alebo „Variant“. Väčšina konkrétnych zraniteľností sú Base alebo Variant.

CWE taktiež poskytuje „View“ (pohľad), čo je skupina zraniteľností v závislosti od domény. Napríklad, poskytuje „View“ pre zraniteľnosti v softvéri a zraniteľnosti v hardvéri.

4 Návrh

4.1 Design



Obr. 4: Prvý návrh aplikácie v nástroji Figma

Okno aplikácie sa skladá zo 6 základných častí:

1. Navigácia – klasická všeobecne zaužívaná navigácia v hornej časti okna – obsahuje všetky základné aj pokročilé funkcie na správu projektov, nastavení a nastavenie samotnej aplikácie.
2. Riadok s nástrojmi – určený pre nástroje používané pri práci s rôznymi časťami aplikácie. Tieto nástroje sa môžu meniť v závislosti od aktuálne otvoreného okna.
3. Navigácia v ľavej časti obrazovky – obsahuje položky **Definition** na definíciu samotného projektu. Tu sa vytvárajú itemy, damage scenarios, threat scenarios, attack paths a prepojenia medzi nimi. **Analysis** slúži na vyhodnotenie rizík. **Resolution** slúži na návrhy riešenia a rôzne iné poznámky.
4. Hlavná časť aplikácie – obsahuje karty, ktoré sa otvárajú kliknutím na navigáciu v ľavej časti obrazovky alebo kliknutím na odkazy v hlavnej časti aplikácie. Obsah kariet je už samotné jadro aplikácie – t. j. napríklad samotné vytváranie itemov, prípadne spojení medzi nimi, vytváranie damage scenarios, výsledky analýzy a podobne.
5. Console – umožňuje používateľovi zadávať príkazy a zobrazuje históriu vykonaných akcií.
6. Details – zobrazuje detaily prvku, ktorý je práve vybraný. Napríklad pri analýze môže používateľ kliknúť na damage scenario a v detailoch by sa mu o ňom zobrazili bližšie informácie. Ak by chcel používateľ všetky informácie k danému prvku alebo by ho chcel modifikovať, mal by mať možnosť prvok jednoducho otvoriť aj ako okno v hlavnej časti aplikácie.

4.2 Definovanie systému

Na analýzu systému je najskôr potrebné definovať jeho časti, dáta, ktoré používajú, a vzťahy medzi nimi. Na to bude možné využiť systémový diagram alebo jednotlivé tabs pre konkrétne položky (komponenty, dátové entity).

4.2.1 Komponenty

Základom všetkého budú komponenty, z ktorých budeme skladat analyzovaný systém. Každý komponent bude mať svoje jedinečné ID, jeho názov a prípadný popis. Tiež môže obsahovať **dátové entity** a využívané **technológie**.

Pre komponenty budú definované vzťahy parent-child, pomocou ktorých bude možné jednoduchšie modelovať a agregovať hrozby na komponentoch. Ďalším vzťahom bude vzťah komunikácie medzi komponentmi, kde jeden komponent môže komunikovať s ďalšími komponentmi. Pri tomto vzťahu je možné pre každý jednotlivý vzťah zdefinovať jeho názov.

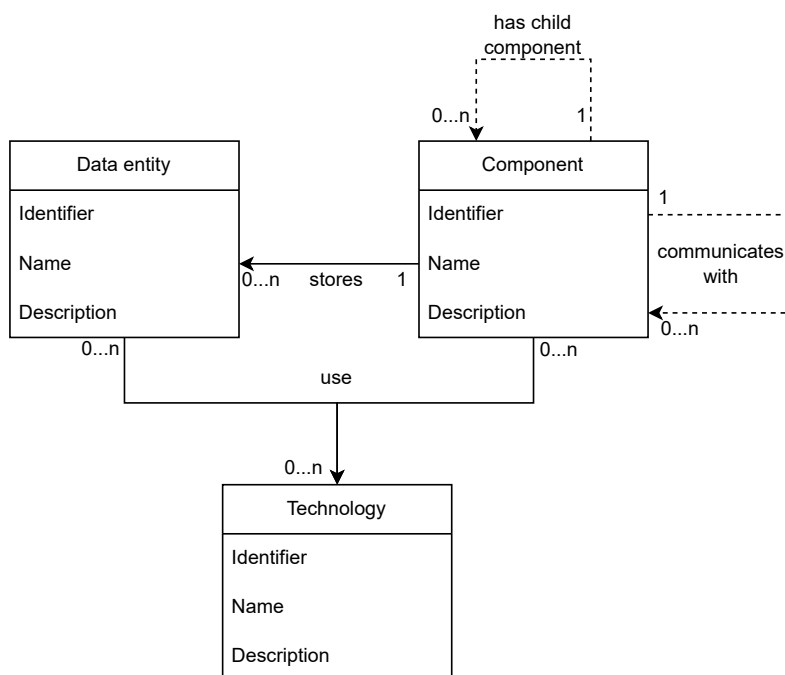
Prvým/základným komponentom bude tzv. systémový komponent, ktorý bude obsahovať všetky komponenty projektu ako children.

4.2.2 Dátové entity

Dátové entity slúžia na reprezentáciu dát, ktoré používajú komponenty. Každá dátová entita bude mať svoje jedinečné ID, názov a prípadný popis dát, ktoré reprezentuje.

4.2.3 Technológie

Technológie budú reprezentovať konkrétne technológie využívané v automobilovom priemysle (TCP/IP, UDP/IP, CAN bus, atď.). Sú definované jedinečným ID, názvom a krátkym popisom. Súčasťou aplikácie bude aj preddefinovaný zoznam týchto technológií, ale používateľ bude mať možnosť definovať vlastné, ak je to potrebné.



4.3 Identifikácia rizík

Ďalšou súčasťou analýzy systému bude proces analýzy rizík, ktorý bude zjednodušený s využitím asistentov, ktorí pomôžu vytvoriť scenáre poškodenia a korešpondujúce scenáre útoku pre jednotlivé dátové entity alebo komponenty.

4.3.1 Asistent scenárov poškodenia

Asistent scenárov poškodenia bude založený na dvoch krokoch. V prvom kroku používateľ prejde všetky komponenty/dátové entity, určí, kde môže vzniknúť prípadná hrozba, a označí, ktorú časť alebo časti CIA to ovplyvňuje.

Po prejdení všetkých komponentov a dátových entít a určení hrozieb sa automaticky vytvoria kostry scenárov poškodenia pre všetky označené komponenty. Používateľ ich následne môže upraviť, napríklad upraviť, aká časť CIA je postihnutá týmto konkrétnym scenárom poškodenia. Tiež môže doplniť položku impact scale, ktorá určuje počet zariadení, ktoré môžu byť zasiahnuté, a položku impact, ktorá definuje vplyv scenára na safety, financial, operational a privacy aspekty.

4.3.2 Asistent scenárov hrozieb

Asistent scenárov hrozieb bude využívať zoznam definovaných hrozieb (budú existovať predvolené hrozby, ale používateľ bude môcť dedefinovať vlastné), ktoré bude posudzovať na komponentoch/dátových entitách, kde boli určené scenáre poškodenia.

Následne budú scenáre hrozieb automaticky priradené k scenárom poškodenia, čím bude mať používateľ zľahčenú prácu. Pre kompletnosť scenára hrozby je potrebné dedefinovať kroky útoku, pomocou ktorých je hrozba realizovaná.

4.3.3 Asistent krokov útoku

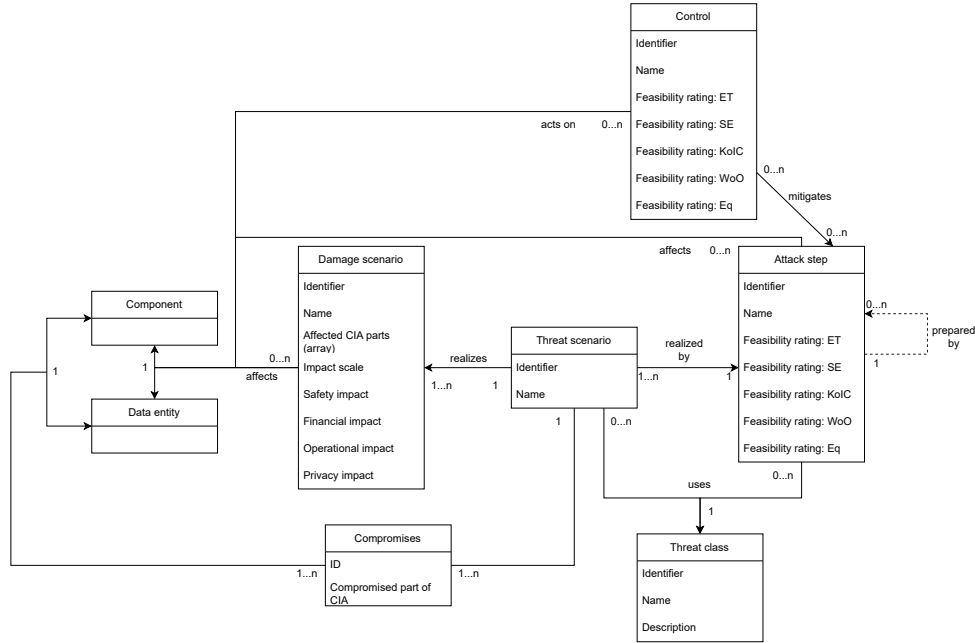
Asistent scenárov tiež využíva zoznam definovaných hrozieb a používateľom definované scenáre hrozieb. Pre každý scenár hrozby je vytvorený krok útoku, ktorý obsahuje informácie o tom, akú hrozbu spôsobuje, na akom komponente/dátovej entite sa nachádza.

Používateľ môže dedefinovať controls, ktoré dokážu zmierniť hrozbu alebo jej predísť, prípadne prípravný attack step, ktorý musí byť vykonaný pre fungovanie tohto kroku. Ďalším krokom bude vyplniť feasibility tabuľku, aby bol správne určený dopad kroku útoku na systém.

4.3.4 Controls

Controls slúžia na definovanie prvkov/opatrení, ktoré dokážu zmeniť dopad krokov útoku. Pri ich definícii je potrebné uviesť názov, komponent alebo dátovú entitu, na ktorú sa aplikujú, a aké scenáre hrozby a kroky útoku mitigujú alebo na ktoré sa aplikujú. Tiež je potrebné zadefinovať feasibility tabuľku, ktorá bude definovať feasibility hodnoty po aplikovaní control.

Podobne ako pri technológiach bude existovať zoznam preddefinovaných controls (napr. symetrické šifrovanie, asymetrické šifrovanie), ktoré bude používateľ môcť použiť.



4.4 Analýza rizík a mitigácia rizík

Na analýzu rizík sa budú používať control skupiny definované používateľom, ktoré umožnia vytvárať rôzne kombinácie opatrení, ktoré bude môcť porovnávať.

Výsledok analýzy je následne možné exportovať ako dokument, ktorý bude obsahovať informácie o systéme, scenároch poškodenia, scenároch hrozieb aj krokoch útoku. Tiež bude obsahovať control opatrenia a spôsob, akým sú aplikované na jednotlivé kroky útoku/scenáre hrozieb, kde bude jasne vidieť, ktoré z nich boli alebo neboli „opravené“ opatreniami.

Ďalším nástrojom na analýzu a mitigáciu rizík bude možnosť použitia LLM, ktorý bude môcť využiť na analýzu alebo pomoc pri definovaní systému.

4.4.1 Control skupiny

Skupiny control opatrení budú definované používateľom, s tým, že budú existovať dve predvolené nastavenia: bez aplikovaných opatrení a s aplikovanými všetkými control opatreniami.

Používateľ bude môcť vytvoriť ďalšie skupiny, kde bude môcť definovať ich meno, prípadný popis a potom bude môcť priradiť jednotlivé control opatrenia.

4.4.2 Využitie LLM

Používateľ bude mať možnosť využiť LLM (buď lokálny model, alebo vie nastaviť využitie modelu cez API poskytovateľa) na prácu s definovaným systémom v aplikácii.

Prvou možnosťou bude využitie nástrojov (implementované pomocou LangChain Tools), ktorými bude možné vytvoriť komponenty/dátové entity napísaním inštrukcií LLM (napr. „vytvor ECU komponent, ktorý obsahuje 3 data entity a má podkomponent...“), čím sa značne zjednoduší proces modelovania systému.

Ďalšou možnosťou bude vykonávať analýzu nad vytvoreným systémom, kde bude mať model (ak to používateľ povie) prístup ku kontextu systému a bude môcť analyzovať jeho časti podľa štandardu.

4.5 Backend

Backend server bol navrhnutý s použitím technológií Django spolu s Django REST Framework (<https://www.django-rest-framework.org/>). Okrem manipulácie s databázou mal server zabezpečovať autorizáciu a vnútornú logiku aplikácie.

Nasledujúci zoznam endpointov je pôvodný návrh API z prototypovej fázy. Nejde o úplnú dokumentáciu finálnej implementácie. Výsledné API sa počas implementácie rozšírilo o projekty, JWT autorizáciu, `project_id`, damage scenario concerns, control groups, risk treatment, cybersecurity goals, MITRE endpointy, report a LLM asistenta. Finálny stav je preto zložitejší a je opísaný neskôr v kapitole „Výsledná implementácia“, najmä v časti „Backend API a synchronizácia vzťahov“.

GET /api/component

```
{component: [
  {id, name, description},
  {id, name, description}
]}
```

POST /api/component

```
{name, description, data_entities: [data_entity_id1, data_entity_id2],
technologies: [technology_id1, technology_id2]}
```

GET /api/component/<id>

```
{id, name, description, data_entity: [
  {id, name, description},
  {id, name, description},
],
technology: [
  {id, name, description},
  {id, name, description}
],
damage_scenario: [
  {id, name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
    operational_impact, privacy_impact, threat_scenario_id},
  {id, name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
    operational_impact, privacy_impact, threat_scenario_id}
],
control: [control_id1, control_id2],
threat_scenarios: [threat_scenario_id1, threat_scenario_id2],
attack_steps: [attack_step_id1, attack_step_id2],
}
```

POST /api/component/<id>

```
{name, description, data_entity: [
  {id, name, description},
]}
```

```

        {id, name, description},
    ],
    technology: [
        {id, name, description},
        {id, name, description}
    ],
    damage_scenario: [
        damage_scenario_id1, damage_scenario_id2
    ],
    control: [control_id1, control_id2],
    threat_scenarios: [threat_scenario_id1, threat_scenario_id2],
    attack_steps: [attack_step_id1, attack_step_id2],
}

```

GET /api/damage__scenario

```

{damage_scenario : [
    {id, name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
      operational_impact, privacy_impact, threat_scenario_id, component_id},
    {id, name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
      operational_impact, privacy_impact, threat_scenario_id, component_id}
]}

```

POST /api/damage__scenario

```

{name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
operational_impact, privacy_impact, threat_scenario_id, component_id}

```

GET /api/damage__scenario/component/<component_id>

(Rovnaké ako GET /api/damage__scenario, ale vyfiltrované pre komponent s daným id)

GET /api/damage__scenario/<id>

```

{id, name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
operational_impact, privacy_impact, threat_scenario_id, component_id}

```

POST /api/damage__scenario/<id>

```

{name, affected_CIA_parts, impact_scale, safety_impact, financial_impact,
operational_impact, privacy_impact, threat_scenario_id, component_id}

```

GET /api/control

```

control: [

```

```

    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, component_id,
      attack_steps: [attack_step_id1, attack_step_id2]},
    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, component_id,
      attack_steps: [attack_step_id1, attack_step_id2]},
  ]

```

POST /api/control

```

{name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, component_id,
  attack_steps: [attack_step_id1, attack_step_id2]}

```

GET /api/control/component/<component_id>

(Rovnaké ako GET /api/control, ale vyfiltrované pre komponent s daným id)

GET /api/control/<id>

```

{id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, component_id,
  attack_steps: [attack_step_id1, attack_step_id2]}

```

POST /api/control/<id>

```

{name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, component_id,
  attack_steps: [attack_step_id1, attack_step_id2]}

```

GET /api/attack_step

```

attack_step: [
  {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by: [
    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by},
    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by}
  ], component_id, threat_class_id, threat_scenario: [
    threat_scenario_id1, threat_scenario_id2
  ], controls: [
    controls_id_1, controls_id_2
  ] },
  {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by: [
    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by},
    {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by}
  ], component_id, threat_class_id, threat_scenario: [
    threat_scenario_id1, threat_scenario_id2
  ], controls: [
    controls_id_1, controls_id_2
  ] },
]

```

POST /api/attack_step

```
{id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by: [
  attack_step_id1, attack_step_id2
], component_id, threat_class_id, threat_scenario: [
  threat_scenario_id1, threat_scenario_id2
], controls: [
  controls_id_1, controls_id_2
] },
```

GET /api/attack_step/component/<component_id>

(Rovnaké ako GET /api/attack_step, ale vyfiltrované pre komponent s daným id)

GET /api/attack_step/<id>

```
{id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by: [
  {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by},
  {id, name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by}
], component_id, threat_class_id, threat_scenario: [
  threat_scenario_id1, threat_scenario_id2
], controls: [
  controls_id_1, controls_id_2
] }
```

POST /api/attack_step/<id>

```
{name, fr_et, fr_se, fr_koC, fr_Wo0, fr_eq, prepared_by: [
  attack_step_id1, attack_step_id2
], component_id, threat_class_id, threat_scenario: [
  threat_scenario_id1, threat_scenario_id2
], controls: [
  controls_id_1, controls_id_2
] }
```

GET /api/threat_scenario

```
{ threat_scenario: [
  {id, name, attack_step: [attack_step_id1, attack_step_id2],
    damage_scenario: [damage_scenario_id1, damage_scenario_id2],
    threat_class_id, compromises: [
      {component_id, compromised_part_cia},
      {component_id, compromised_part_cia}]}],
  {id, name, attack_step: [attack_step_id1, attack_step_id2],
    damage_scenario: [damage_scenario_id1, damage_scenario_id2],
    threat_class_id, compromises: [
      {component_id, compromised_part_cia},
      {component_id, compromised_part_cia}]}}
```

```
}}
```

POST /api/threat_scenario

```
{name, attack_step: [attack_step_id1, attack_step_id2],  
damage_scenario: [damage_scenario_id1, damage_scenario_id2],  
threat_class_id, compromises: [  
  {component_id, compromised_part_cia},  
  {component_id, compromised_part_cia}]},  
]}
```

POST /api/threat_scenario/component/<component_id>

(Rovnaké ako GET /api/threat_scenario, ale vyfiltrované pre komponent s daným id)

GET /api/threat_scenario/<id>

```
{id, name, attack_step: [attack_step_id1, attack_step_id2],  
damage_scenario: [damage_scenario_id1, damage_scenario_id2],  
threat_class_id, compromises: [  
  {component_id, compromised_part_cia},  
  {component_id, compromised_part_cia}]}
```

POST /api/threat_scenario/<id>

```
{name, attack_step: [attack_step_id1, attack_step_id2],  
damage_scenario: [damage_scenario_id1, damage_scenario_id2],  
threat_class_id, compromises: [  
  {component_id, compromised_part_cia},  
  {component_id, compromised_part_cia},  
]}
```

5 Frontend

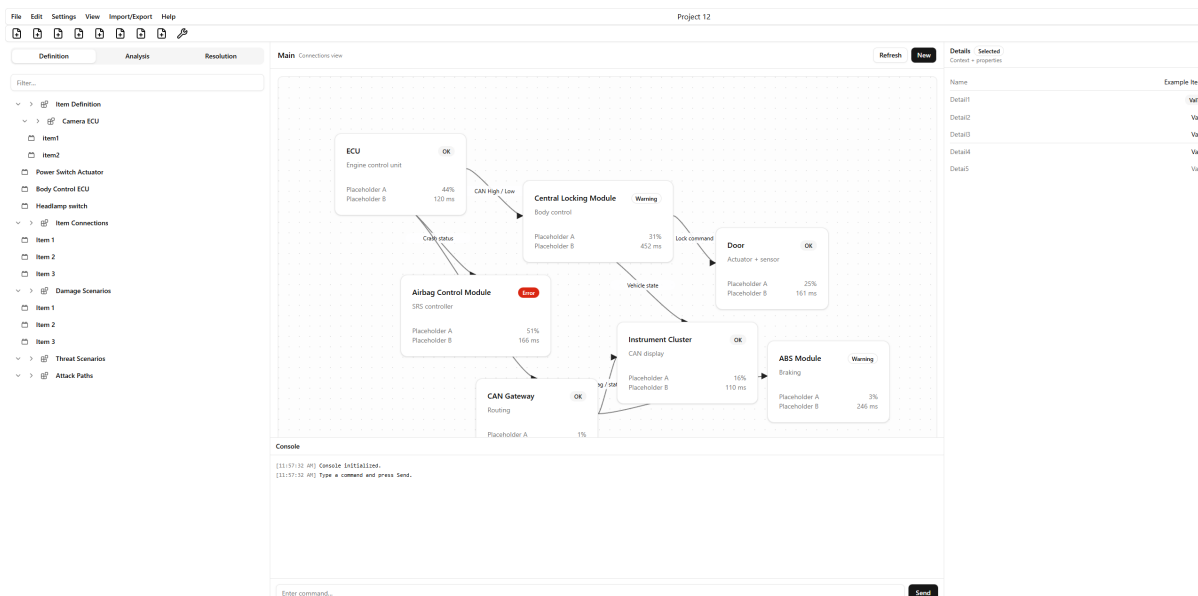
5.1 Návrh a prototyp aplikácie

Na vytvorenie frontendu vyvíjanej aplikácie sme sa rozhodli použiť technológiu React v kombinácii s nástrojom Vite ako vývojovým bundlerom. Technológie volíme prevažne kvôli tomu, že sú moderné, široko používané a poskytujú rýchle vývojové prostredie. React umožňuje rozdeľovanie GUI na menšie logické segmenty, čím sa výrazne uľahčuje vývoj.

Pri vývoji využívame doplnkové knižnice, najmä shaden, Tailwind a Lucide, pomocou ktorých vytvárame konzistentné, moderné a vizuálne zjednotené prostredie. Pri návrhu homepage sme sa inšpirovali dizajnom z nástroja Figma zobrazeného na Prvý návrh aplikácie v nástroji Figma.

V aktuálnej fáze projektu bol vytvorený funkčný prototyp frontendovej časti aplikácie. Prototyp obsahuje základné rozloženie homepage, rozloženie hlavných komponentov a demonštračné dáta, ktoré slúžia ako zástupné hodnoty. Na obrázku nižšie je uvedený screenshot aktuálneho stavu

prototypu aplikácie, ktorý zobrazuje základné rozhranie a štruktúru frontendovej časti. (Prototyp Frontendovej časti)



Obr. 5: Prototyp Frontendovej časti

Pôvodný frontend a backend vznikali ako samostatné prototypy. Vo finálnej fáze boli zlúčené do jedného repozitára, ktorý obsahuje adresáre `frontend/`, `backend/` a `documentation/`.

6 Backend – druhá iterácia

V tejto historickej iterácii prototypu boli dokončené vtedajšie endpointy, boli pridané projekty a CRUD operácie pre hlavné databázové objekty. Tiež bola pridaná autentizácia a autorizácia používateľa pomocou JWT.

6.1 Autentizácia a autorizácia

Používateľ bude mať možnosť si vytvoriť účet, a ku svojmu účtu bude mať následne priradené projekty. V rámci projektov potom už bude mať jednotlivé komponenty, damage scenaria a ostatné. V prvom kroku si používateľ vytvorí účet pomocou nasledovného API callu:

```
POST /api/register/
Content-Type: application/json
{
  "username": "Oliver",
  "password": "heslo123",
  "email": "oliver@example.com"
}
```

Ako odpoveď je z backendu odoslané potvrdenie o vytvorení používateľa. Keď už používateľ má účet, môže sa prihlásiť pomocou nasledovného API callu:

```
POST /api/token/
Content-Type: application/json
{
```

```
"username": "john_doe",  
"password": "securepass123"  
}
```

Odpoveďou je JSON Web Token (poznámka: namiesto <JWT> bude v praxi base64 enkódovaný string):

```
{  
  "access": <accessJWT>,  
  "refresh": <refreshJWT>  
}
```

Tento token následne používateľ odosiela v každej požiadavke v hlavičke:

Authorization: Bearer <accessJWT>

Na základe tohto tokenu sa overí, či má prístup ku projektu, ktorého sa požiadavka týka. V prípade že používateľovi token vyprší, môže si ho obnoviť použitím nasledujúceho API callu:

```
POST /api/token/refresh/  
Content-Type: application/json  
{  
  "refresh": <refreshJWT>  
}
```

6.2 Správa projektov

S autentizáciou a autorizáciou úzko súvisí správa projektov. Po prihlásení by mal používateľ vidieť zoznam svojich projektov. Akonáhle si vyberie jeden z nich, všetky nasledovné požiadavky sa budú týkať iba tohto projektu. V praxi je to vyriešené tak, že novou súčasťou každej požiadavky je položka `project_id`, ktorá určuje, akého projektu sa požiadavka týka. V kombinácii s hlavičkou **Authorization** je teda zaručené, že každý call sa týka práve jedného projektu, pričom na strane servera je overené, že daný používateľ má k tomuto projektu skutočne prístup. Pre vytvorenie projektu používateľ použije nasledujúci API call:

```
POST /api/projects/  
Authorization: <accessJWT>  
Content-Type: application/json  
{  
  "name": "My Risk Assessment Project",  
  "description": "Assessing security risks for main system"  
}
```

Zoznam svojich projektov získa nasledovne:

```
GET /api/projects/  
Authorization: <accessJWT>
```

odpoveď:

```
[  
  {  
    "id": 1,  
    "name": "My Risk Assessment Project",  
    "description": "Assessing security risks for main system",  
    "created_at": "2024-03-22T10:30:00Z"  
  },  
]
```

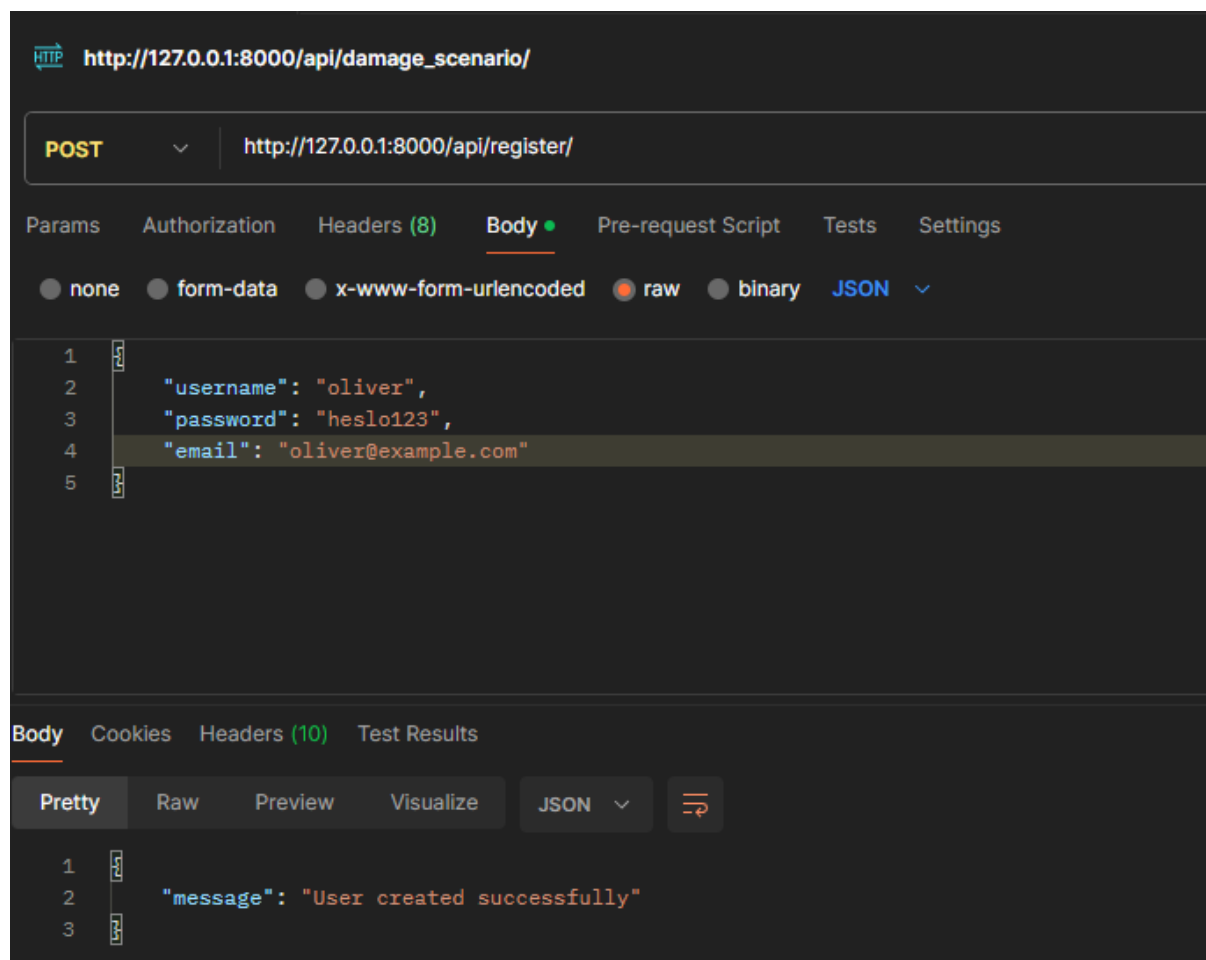
```
{
  "id": 2,
  "name": "Another Project",
  "description": "Security review for new feature",
  "created_at": "2024-03-22T11:00:00Z"
}
```

Následne používa API volania definované v predošlých kapitolách, pričom v každom uvedie ID projektu, s ktorým práve pracuje.

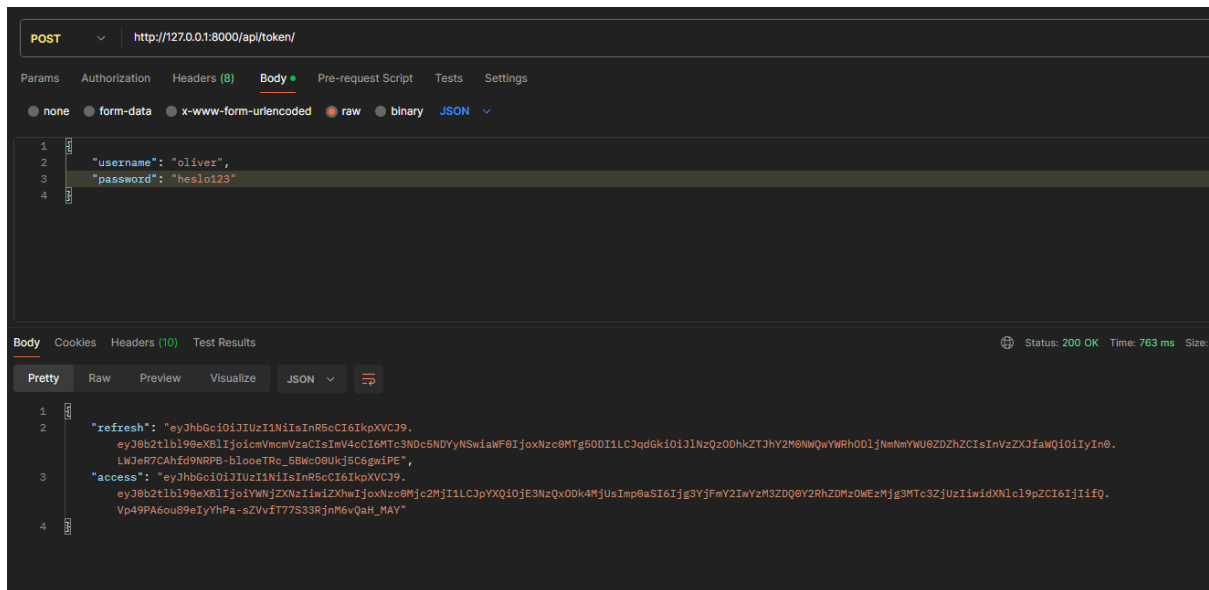
6.3 Príklad použitia backendu

Nasleduje príklad použitia backendu, vrátane registrácie používateľa, s použitím softvéru Postman:

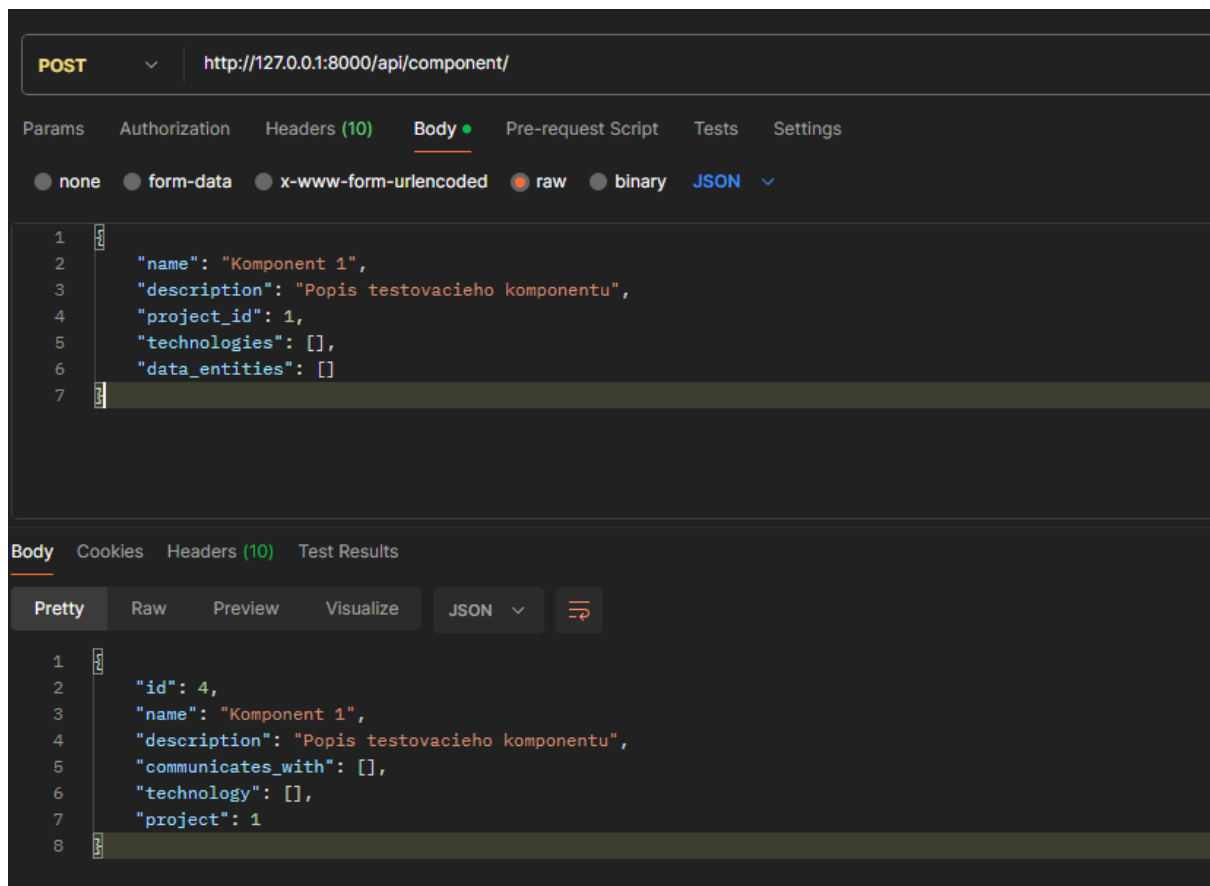
1. Registrácia používateľa



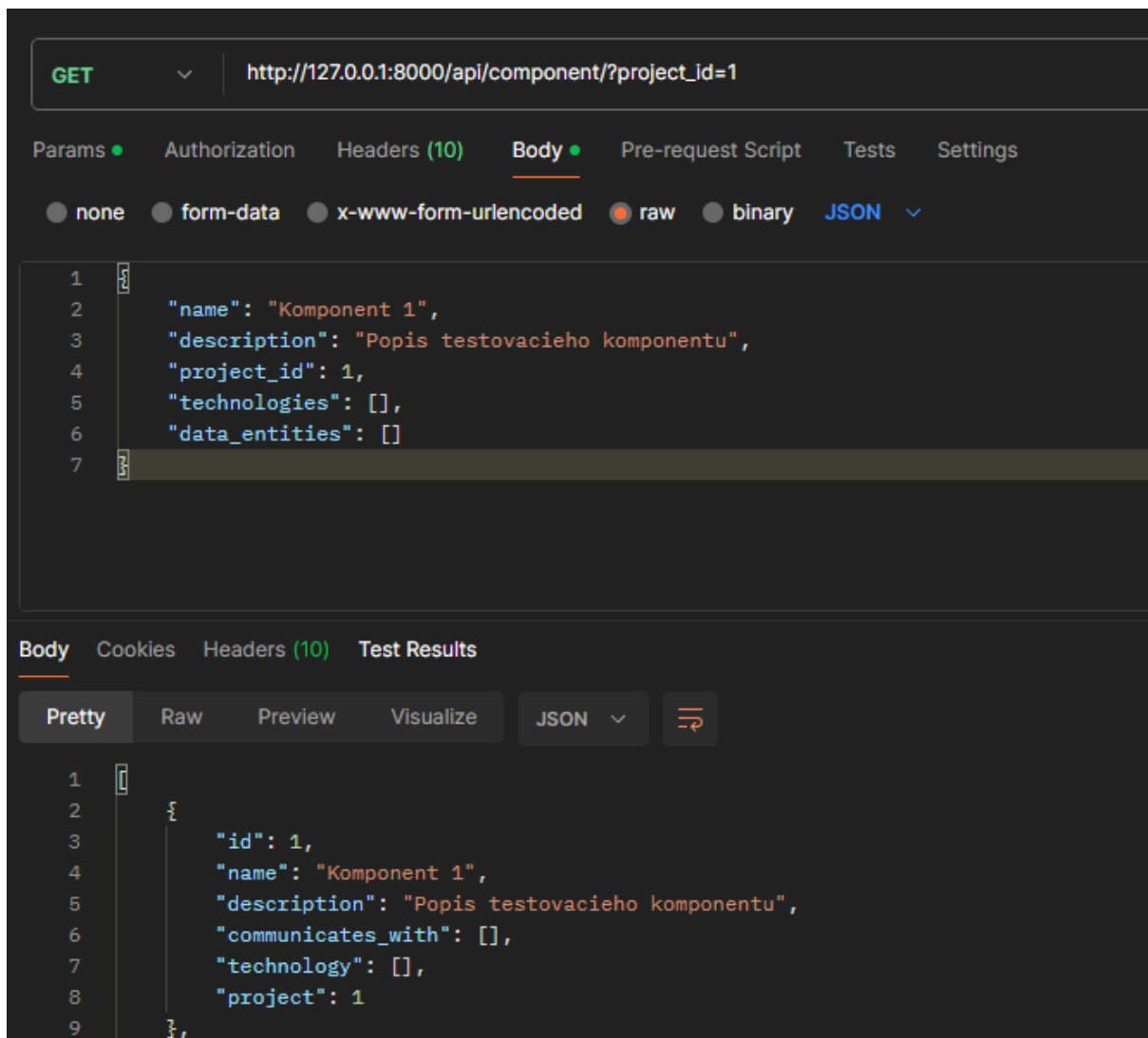
2. Prihlásenie používateľa – v odpovedi je token, ktorý sa použije v každej ďalšej požiadavke



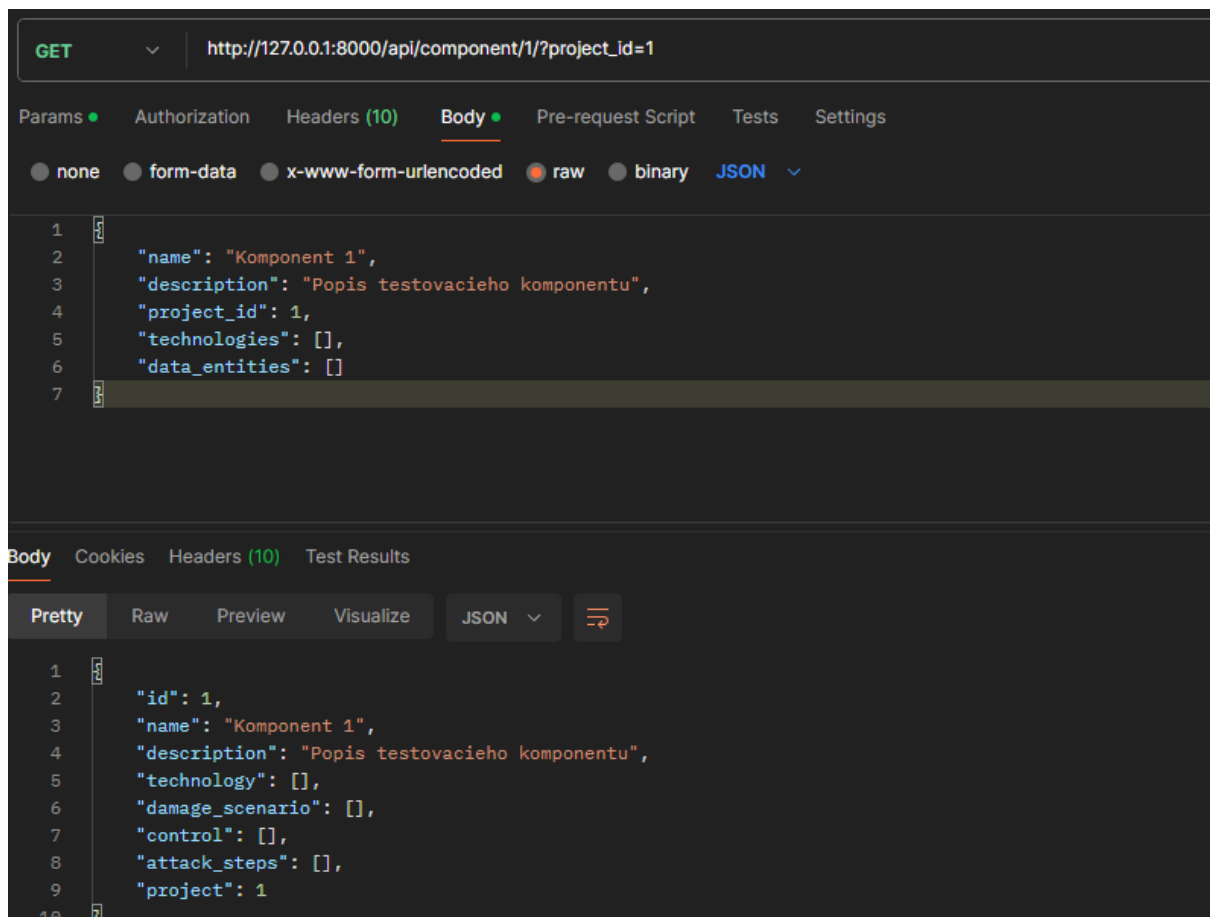
3. Vytvorenie projektu. Na druhom obrázku je zobrazené, že token získaný v minulom kroku je v hlavičke **Authorization**. Bude sa odosielať po celý zvyšok relácie. Vytvorený projekt je takto asociovaný s daným používateľom a akékoľvek operácie s projektom budú musieť byť iba od tohto používateľa.



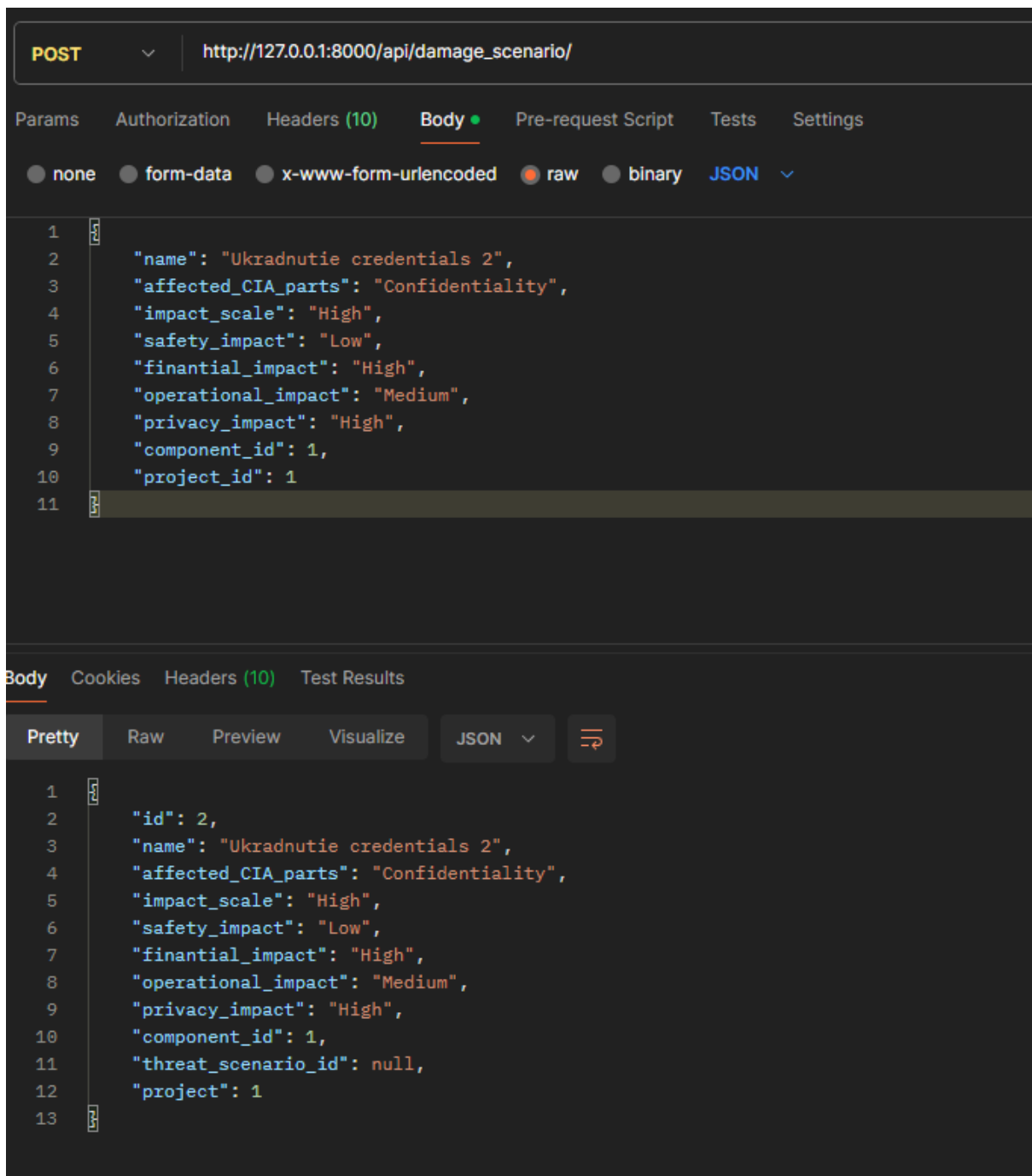
5. Získanie zoznamu komponentov. V prípade GET požiadavky je číslo projektu v query parametroch (telo požiadavky na tomto obrázku je bezpredmetné, keďže ide o GET požiadavku).



6. Získanie detailov konkrétneho komponentu. V tomto prípade je v query URL číslo komponentu (tak ako v predošlej verzii API) a následne je v query parametroch číslo projektu (tak ako v predošlej požiadavke)



7. Vytvorenie damage scenaria. Súčasťou tela požiadavky je `component_id` a `project_id`. Daný damage scenario je tak asociovaný s práve jedným komponentom, ktorý je v práve jednom projekte, pričom ako už bolo spomenuté, vlastníctvo daného projektu používateľom je zabezpečené hlavičkou **Authorization**. Na rovnakom princípe fungujú aj ostatné POST operácie, ako vytvorenie attack path alebo threat scenario.



8. Získanie detailov konkrétneho komponentu. Tento krok je rovnaký ako pred dvoma krokmi – je tu iba pre ukážku, akým spôsobom sa damage scenario vytvorený v predošlom kroku pridal do komponentu.

```
GET http://127.0.0.1:8000/api/component/1/?project_id=1

Params • Authorization Headers (10) Body • Pre-request Script Tests Settings

Query Params

Key

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON ↕

1  [
2    {
3      "id": 1,
4      "name": "Komponent 1",
5      "description": "Popis testovacieho komponentu",
6      "technology": [],
7      "damage_scenario": [
8        {
9          "id": 1,
10         "name": "Ukradnutie credentials",
11         "affected_CIA_parts": "Confidentiality",
12         "impact_scale": "High",
13         "safety_impact": "Low",
14         "finantial_impact": "High",
15         "operational_impact": "Medium",
16         "privacy_impact": "High",
17         "component_id": 1,
18         "threat_scenario_id": null,
19         "project": 1
20       },
21     ],
22   },
23   {
24     "id": 2,
25     "name": "Ukradnutie credentials 2",
26     "affected_CIA_parts": "Confidentiality",
27     "impact_scale": "High",
28     "safety_impact": "Low",
29     "finantial_impact": "High",
30     "operational_impact": "Medium",
31     "privacy_impact": "High",
32     "component_id": 1,
33     "threat_scenario_id": null,
34     "project": 1
35   }
36 ]
```

Tieto backendové zmeny boli neskôr zlúčené s frontendovou časťou do spoločného repozitára projektu.

7 Výsledná implementácia

Výsledkom projektu je funkčná webová aplikácia na tvorbu TARA analýzy pre automobilový systém. Aplikácia nie je iba jednoduchý formulár nad databázou, ale spája viacero spôsobov práce nad rovnakým modelom: grafický pohľad, tabuľkový pohľad, detailový editor, asistentov, konzolu s LLM návrhmi, výpočet rizík, prácu s control skupinami a export výsledného reportu do PDF. Hlavná myšlienka ostala rovnaká ako v návrhu – používateľ najprv vytvorí model systému, potom nad ním definuje damage scenarios, threat scenarios, attack steps a controls, a nakoniec vie vyhodnotiť riziká a pripraviť podklady pre mitigáciu.

Finálna implementácia je rozdelená na dve časti. Backend je vytvorený v technológii Django spolu s Django REST Framework. Stará sa o databázu, autorizáciu, kontrolu prístupu k projektom, výpočty TARA hodnôt, generovanie reportu a komunikáciu s LLM asistentom. Frontend je vytvorený v technológii React s nástrojom Vite. Na rozhranie používa knižnice Tailwind, shadcn/ui, lucide-react, Zustand a @xyflow/react. Frontend teda rieši hlavne prácu s používateľom, vizualizáciu vzťahov, editáciu modelov a volanie backend API.

V nasledujúcich častiach je opísaná výsledná aplikácia tak, ako funguje po dokončení. Popis je písaný z pohľadu práce používateľa, ale pri každej väčšej funkcionalite je zároveň uvedené, ako je riešená vnútri aplikácie a ako sú jednotlivé časti medzi sebou prepojené.

7.1 Beh a nasadenie aplikácie

Projekt je možné spustiť dvoma spôsobmi. Pri vývoji je výhodné spúšťať frontend a backend oddelene: Django beží na porte 8000 a Vite vývojový server na porte 5173. V tomto režime frontend automaticky volá API na adrese `http://localhost:8000/api`.

Pre reálne nasadenie je pripravený jeden Docker kontajner. Docker build najprv zostaví React/Vite frontend poskytovaný pod cestou `/static/`, potom výsledný `dist` adresár skopíruje do výsledného image. Django následne poskytuje statické frontend súbory cez `/static/` a samotnú single-page aplikáciu na koreňovej adrese `/`. Backend API ostáva dostupné pod `/api/` a Django admin pod `/admin/`. Potrebná Docker Compose konfigurácia preto obsahuje iba službu `app` a volume pre SQLite databázu.

7.2 Celkový tok práce v aplikácii

Po otvorení aplikácie sa používateľ najprv prihlási. Prihlásenie vyžaduje meno a heslo. Ak používateľ ešte nemá účet, môže na tej istej obrazovke prepnúť jednoduchý režim vytvorenia nového používateľa. Po registrácii sa frontend rovno pokúsi prihlásiť s novým účtom. Po úspešnom prihlásení backend vráti dvojicu JWT tokenov – `access` a `refresh`. Frontend si ich uloží do `sessionStorage` a následne ich používa pri každej požiadavke na backend.

Po prihlásení používateľ vyberie existujúci projekt alebo vytvorí nový projekt. Projekt predstavuje samostatný TARA workspace. Všetky komponenty, technológie, scenáre, kroky útoku, controls a výsledné riziká sú vždy viazané na konkrétny projekt. Táto väzba je dôležitá aj bezpečnostne, pretože backend pri každom API volaní overuje, či prihlásený používateľ skutočne vlastní projekt, ktorého sa požiadavka týka. Nestačí teda poznať `project_id`; požiadavka musí mať aj platný JWT token pre vlastníka projektu.

Po výbere projektu sa otvorí hlavná pracovná plocha. Tá je rozdelená na tri hlavné režimy:

- **Graph View** – grafický pohľad, kde sú všetky modelové objekty zobrazené ako karty a vzťahy medzi nimi ako spojenia.
- **Table View** – tabuľkový pohľad vhodný na rýchlu kontrolu scenárov, attack steps, controls, rizík a cybersecurity goals.

- **Assistants** – asistenti, ktorí pomáhajú dopĺňať TARA model, napríklad identifikovať assets a vytvoriť základné damage/threat scenáre.

Tieto pohľady nepracujú s oddelenými dátami. Všetky používajú rovnaký stav projektu načítaný z backendu. Vo frontende je tento stav uložený v globálnom **Zustand** store. Store obsahuje mapy pre technológie, komponenty, data entities, controls, threat classes, attack steps, threat scenarios, damage scenarios, compromises a cybersecurity goals. Keď používateľ niečo upraví v detailovom paneli alebo cez asistenta, zmena sa odošle do backendu a následne sa obnoví aj lokálny stav. Vďaka tomu sa rovnaká zmena prejaví v grafe, tabuľkách aj v detailoch.

Rozhranie hlavnej aplikácie má stabilné rozloženie. Hore je menubar a toolbar, vľavo je navigácia podľa aktuálneho režimu, v strede hlavný pracovný priestor, dole je konzola a vpravo detailový panel. V hornom menu je okrem položiek **File** a **Help** aj tlačidlo **Login**, ktoré vymaže aktuálnu session, project ID a tokeny, a tým vráti aplikáciu späť na prihlasovaciu obrazovku. Toto je najjednoduchší spôsob, ako počas behu aplikácie prepnúť používateľa. Toto rozloženie nadväzuje na pôvodný návrh aplikácie z kapitoly „Návrh“, ale vo výslednej implementácii sú jednotlivé časti už funkčne prepojené.

7.3 Autentizácia, projekty a prístupové práva

Autentizácia je implementovaná pomocou JWT tokenov cez knižnicu `django-rest-framework-simplejwt`. Backend poskytuje endpointy `/api/register/`, `/api/token/` a `/api/token/refresh/`. Registrácia vytvorí nového používateľa v štandardnom Django modeli **User**. Vo frontende je registrácia dostupná priamo z prihlasovacej obrazovky cez možnosť „Create a new user“. Používateľ zadá meno, heslo a prípadne email. Po úspešnej registrácii frontend zavolá bežné prihlásenie, takže používateľ nemusí manuálne prechádzať späť na login. Prihlásenie vráti access token a refresh token. Frontend access token posíla v hlavičke:

Authorization: Bearer <accessJWT>

Každý projekt je uložený v modeli **Project**. Má názov, popis, vlastníka a dátum vytvorenia. Model projektu je jednoduchý, ale je centrálnym bodom celej aplikácie, pretože všetky ostatné projektové objekty majú na projekt cudzie kľúče alebo sú naň nepriamo naviazané.

Backend používa pomocnú funkciu `check_project_access`. Tá sa volá v jednotlivých endpointoch a kontroluje, či existuje projekt s daným ID a či jeho **owner** je aktuálne prihlásený používateľ. Ak projekt neexistuje alebo nepatrí danému používateľovi, backend vráti chybu 403. Toto je použité konzistentne pri komponentoch, scenároch, control skupinách, rizikách, reportoch aj LLM asistentovi.

Vo frontende sa projekt vyberá na stránke **ProjectSelector**. Tá po prihlásení načíta zoznam projektov cez `/api/projects/`. Používateľ môže projekt vybrať alebo rovno vytvoriť nový. ID a meno vybraného projektu sa uložia do `sessionStorage`. Po otvorení projektu komponent **ProjectPage** zavolá `loadProjectState`, čím sa naraz načíta celý projektový model a vypočítané riziká.

Toto riešenie má praktickú výhodu: používateľ nemusí v jednotlivých obrazovkách opakovane vyberať projekt. Frontend automaticky prikladá `project_id` ku každému relevantnému API volaniu. Backend napriek tomu nikdy neverí iba frontendu, ale vlastníctvo projektu kontroluje sám.

7.4 Dátový model výslednej aplikácie

Finálna databázová vrstva je postavená okolo TARA pojmov. Oproti pôvodnému návrhu pribudli niektoré objekty a niektoré vzťahy boli upravené tak, aby lepšie podporovali grafický editor a

výpočet rizík.

7.4.1 Technológie

Model **Technology** reprezentuje technológie použité v systéme. Technológia má názov, popis a projekt. K technológiám sa môžu pripájať komponenty aj data entities. V praxi sem patria napríklad CAN, Ethernet, Bluetooth, Cellular, FreeRTOS, AUTOSAR SecOC alebo kryptografické mechanizmy.

Technológie samy o sebe netvorí riziko, ale pomáhajú opísať prostredie, v ktorom komponent funguje. V LLM kontexte a v asistentoch sa používajú ako nápoveda pre to, aké typy hrozieb alebo útokov môžu byť relevantné. Vo frontende sa technológie zobrazujú v strome pod komponentmi, v grafickom pohľade ako samostatné karty a v detailoch ako vzťahy.

7.4.2 Komponenty

Model **Component** reprezentuje časti analyzovaného systému. Komponent má názov, popis, projekt, zoznam použitých technológií a vzťah **communicates_with** na iné komponenty. Tento komunikačný vzťah je obojsmerný **ManyToMany** vzťah, ktorý slúži na modelovanie toho, ktoré komponenty medzi sebou komunikujú.

Komponent je v aplikácii jeden z najdôležitejších modelov, pretože veľká časť TARA objektov sa naň naväzuje. Data entity patria ku komponentom, attack steps sa vykonávajú na komponentoch, controls sa aplikujú na komponenty a threat scenarios môžu obsahovať viacero komponentov. Backend má pri komponente aj odvodené vlastnosti **mapped_threat_scenarios** a **mapped_damage_scenarios**. Tie nezapisujú nové dáta do databázy, ale vypočítavajú, ktoré threat scenarios a damage scenarios sú s komponentom spojené cez existujúce vzťahy.

7.4.3 Data entities

Model **DataEntity** slúži na reprezentáciu dát, ktoré majú v systéme hodnotu. Data entity má názov, popis, voliteľný komponent, zoznam technológií a projekt. V aktuálnej implementácii sa data entities používajú hlavne ako assets pri asistente Asset Identification a ako objekty načítané cez detail komponentu.

V pôvodnom návrhu boli data entities samostatným pilierom definície systému. Vo výslednej implementácii sú podporené v dátovom modeli a v častiach frontendu, ale nemajú samostatné plnohodnotné CRUD endpointy ako komponenty, scenáre alebo controls. Samotná TARA analýza je preto vedená najmä cez komponenty, damage scenarios, threat scenarios a attack steps.

7.4.4 Threat classes

Model **ThreatClass** reprezentuje triedu hrozby. Má názov, popis, voliteľné MITRE tactic ID a meno taktiky. Threat class sa dá priradiť k attack stepu alebo threat scenariu. Tým vzniká jednoduché prepojenie medzi vlastným TARA modelom a externou MITRE terminológiou.

Na strane backendu sú k dispozícii aj endpointy `/api/mitre/tactics/` a `/api/mitre/techniques/`. Tieto endpointy získavajú dáta z MITRE ATT&CK STIX zdroja a poskytujú ich frontendu. Vo formulári pre threat class a attack step sa tak dajú používať MITRE taktiky a techniky bez toho, aby ich používateľ musel manuálne prepisovať.

7.4.5 Attack steps

Model **AttackStep** popisuje konkrétny krok útoku. Má názov, popis, požadovaný prístup, komponent, threat class, MITRE technique ID a názov techniky. Dôležitou časťou sú hodnoty

`fr_et`, `fr_se`, `fr_koC`, `fr_WoD` a `fr_eq`. Tieto hodnoty reprezentujú faktory attack feasibility:

- **Elapsed Time** – čas potrebný na prípravu alebo vykonanie útoku.
- **Specialist Expertise** – úroveň odbornosti útočníka.
- **Knowledge of Component** – úroveň znalosti komponentu alebo systému.
- **Window of Opportunity** – dostupnosť príležitosti na útok.
- **Equipment** – potrebné vybavenie.

Attack step môže mať predchádzajúce kroky cez vzťah `previous_steps`. Tento vzťah je orientovaný a umožňuje vytvoriť attack path. Ak napríklad útok vyžaduje najprv získanie fyzického prístupu a až potom flashovanie firmware, druhý krok môže mať prvý krok ako `previous_step`. Backend pri výpočte threat scenaria potom nevníma attack steps iba ako nezávislý zoznam, ale snaží sa z nich vytvoriť možné cesty.

7.4.6 Threat scenarios

Model `ThreatScenario` spája komponenty, attack steps a damage scenarios. Threat scenario odpovedá na otázku, aká hrozba môže viesť k damage scenariu a cez ktoré attack steps sa dá realizovať. Má názov, popis, zoznam komponentov, zoznam attack steps, threat class a zoznam damage scenarios.

V implementácii je threat scenario veľmi dôležitý spojovací objekt. Ak attack step nie je priradený k threat scenariu, nevstupuje do výpočtu rizika. Ak damage scenario nie je priradený k threat scenariu, tiež nevzniká risk row. Preto backend aj frontend pri vytváraní vzťahov často automaticky dopĺňajú opačné väzby. Napríklad ak sa attack step pripojí k threat scenariu, backend zároveň vie zabezpečiť, aby komponent attack stepu bol pridaný medzi komponenty threat scenaria.

Threat scenario má aj súvisiaci model **Compromises**. V aplikácii funguje ako compromise záznam. Ukladá komponent, threat scenario a CIA bitmask toho, čo je na danom komponente kompromitované. Vďaka tomu môže threat scenario presnejšie opísať, či ide o stratu Confidentiality, Integrity alebo Availability na konkrétnej časti systému.

7.4.7 Damage scenarios a concerns

Model `DamageScenario` opisuje následok úspešného útoku. Má názov, popis, CIA bitmask, štyri impact hodnotenia a projekt. Štyri impact hodnotenia sú:

- **safety_impact**
- **financial_impact**
- **operational_impact**
- **privacy_impact**

V modeli je pole `impact_scale`, ale vo výslednej implementácii sa nastavuje automaticky v metóde `save`. Hodnota `impact_scale` je maximum zo štyroch impact kategórií. To znamená, že používateľ nemusí ručne udržiavať dve konzistentné hodnoty. Stačí nastaviť safety, financial, operational a privacy impact a backend vypočíta celkový impact level.

CIA je uložená ako 3-bitová hodnota. Bit 4 znamená Confidentiality, bit 2 znamená Integrity a bit 1 znamená Availability. Kombinácie sú súčtom týchto hodnôt. Napríklad hodnota 7 znamená CIA, hodnota 6 znamená C+I a hodnota 1 znamená iba Availability. Backend aj frontend majú pomocné funkcie na prevod tejto hodnoty na čitateľný text.

Dôležitým rozšírením sú **DamageScenarioConcern**. Damage scenario už nemusí mať iba jednu globálnu CIA hodnotu. Concern hovorí, ktorého komponentu sa damage scenario týka a ktorá CIA časť je na ňom zasiahnutá. Backend povoľuje concerns iba pre komponenty, ktoré sú dostupné cez threat scenarios pripojené k danému damage scenariu. Ak sa zmenia vzťahy threat scenaria, backend neplatné concerns odstráni. Tým sa zabráňuje tomu, aby damage scenario odkazovalo na komponent, ktorý už s ním logicky nesúvisí.

Súhrnná CIA hodnota na damage scenariu sa synchronizuje z concerns. Ak má damage scenario viac concerns, backend spraví bitový súčet všetkých CIA častí. Používateľ teda môže v detailovom paneli označiť napríklad Confidentiality na jednom komponente a Integrity na druhom komponente, a damage scenario bude mať súhrnne C+I.

7.4.8 Controls a control classes

Model **Control** reprezentuje bezpečnostné opatrenie. Control má názov, popis, voliteľnú control class, attack feasibility faktory, komponent, attack steps, threat scenarios a projekt. Na rozdiel od attack stepu tieto faktory nepopisujú útok samotný, ale dodatočnú náročnosť, ktorú control pridáva útočníkovi. Ak control napríklad vyžaduje obídienie kryptografickej autentizácie alebo získanie špeciálneho vybavenia, jeho hodnoty sa pripočítajú k hodnotám attack stepu.

Model **ControlClass** slúži ako šablóna pre controls. Obsahuje názov, popis a defaultné hodnoty faktorov. V databáze sú seedované preddefinované control classes. Vo formulári pre control si používateľ môže vybrať triedu, čím sa automaticky vyplnia faktorové polia. Hodnoty sa však dajú ďalej upraviť, takže class slúži ako štartovací bod, nie ako pevné obmedzenie.

Controls môžu byť pripojené k attack steps aj k threat scenarios. Väzba na attack steps ovplyvňuje výpočet efektívnej attack feasibility. Väzba na threat scenarios slúži najmä na traceability, aby bolo jasné, ktorú hrozbu control adresuje. Vo formulári je to aj explicitne uvedené – väzba na threat scenarios sama o sebe nemení výpočty.

7.4.9 Control groups

Model **ControlGroup** umožňuje vytvárať kombinácie controls. Skupina má názov, popis, projekt a zoznam controls. Aplikácia má aj dve virtuálne skupiny, ktoré nie sú uložené v databáze:

- **All Controls** – všetky controls v projekte sú aktívne.
- **No Controls** – žiadne controls nie sú aktívne, zobrazuje sa základný stav bez mitigácie.

Používateľ môže vytvoriť vlastnú skupinu, napríklad „Minimal controls“, „Production controls“ alebo „Proposed controls“. Vybraná skupina je uložená v **sessionStorage** a frontend ju používa pri zobrazovaní efektívnych AFL a RL hodnôt. Backend report zároveň obsahuje porovnanie control scenárov, kde pre každý risk ukazuje základný stav a stav pre jednotlivé control groups.

7.4.10 Risk treatment a cybersecurity goals

Model **RiskTreatment** ukladá rozhodnutie pre dvojicu threat scenario + damage scenario. Podporované rozhodnutia sú **avoid**, **reduce**, **share** a **accept**. Okrem rozhodnutia obsahuje aj textové odôvodnenie. V tabuľke risks vie používateľ priamo otvoriť detail rizika a zapísať treatment podľa ISO/SAE 21434.

Model **CybersecurityGoal** reprezentuje cybersecurity goal. Má názov, popis, voliteľnú hodnotu CAL od 1 do 4, zoznam damage scenarios, zoznam controls a projekt. V aplikácii sa tým uzatvára stopa od damage scenaria cez riziko až po bezpečnostný cieľ a opatrenia, ktoré ho adresujú.

7.5 Výpočet attack feasibility, impact a risk level

Výpočtová logika je oddelená v súbore `calculations.py`. Táto časť je dôležitá, lebo z modelovaných objektov robí TARA analýzu.

Attack feasibility sa počíta z piatich faktorov attack stepu:

```
points = fr_et + fr_se + fr_koC + fr_Wo0 + fr_eq
```

Ak je `fr_et` alebo `fr_Wo0` nastavené na 99, výsledok sa berie ako „Not practical“. Vtedy sa attack potential body nezobrazujú ako číslo a AFL sa nastaví na veľmi nízku uskutočniteľnosť útoku.

Inak sa body mapujú na attack potential a AFL:

- 0 až 9 bodov: **Basic**, AFL **High**, hodnota 5.
- 10 až 13 bodov: **Enhanced-Basic**, AFL **High**, hodnota 4.
- 14 až 19 bodov: **Moderate**, AFL **Medium**, hodnota 3.
- 20 až 24 bodov: **High**, AFL **Low**, hodnota 2.
- viac ako 24 bodov: **Beyond High**, AFL **Very Low**, hodnota 1.

Táto logika je použitá pre attack steps aj pre controls. Pri attack stepe hodnoty reprezentujú náročnosť samotného útoku. Pri controle hodnoty reprezentujú dodatočnú náročnosť na obídenie daného opatrenia.

Impact level sa počíta ako maximum zo safety, financial, operational a privacy impact hodnoty. Hodnoty sú uložené ako číselné voľby:

- 0 – Negligible
- 1 – Moderate
- 2 – Major
- 3 – Severe

Risk level sa počíta z impact level a AFL hodnoty pomocou matice `RISK_LEVEL_MATRIX`. Výsledkom je hodnota 1 až 5. Ak threat scenario nemá žiadny attack step, AFL nie je známe a risk level sa nedá vypočítať.

Pri threat scenarii sa attack feasibility nepočíta iba jednoduchým minimom alebo maximom všetkých attack steps. Backend najprv vytvorí možné attack paths z `previous_steps`. Ak kroky tvoria orientovaný graf, vyhľadá všetky cesty od koreňových krokov po listy. Každá cesta je obmedzená svojím najťažším krokom, teda bottleneckom. Útočník si potom vyberá najľahšiu kompletnú cestu. Implementačne sa pre každú cestu vyberie najhoršie hodnotený krok a následne sa z týchto bottleneckov vyberie pre útočníka najvýhodnejší. Ak kroky nemajú poradie, každý krok sa berie ako samostatná cesta.

Pri aktívnych controls sa používa podobná logika, ale pre každý attack step sa najprv vypočíta efektívna attack feasibility. Frontend aj backend berú relevantné iba tie controls, ktoré sú pripojené k danému attack stepu. Ich hodnoty sa pripočítajú k základným hodnotám attack stepu. Tým sa vie ukázať rozdiel medzi základným rizikom a rizikom po aplikovaní control skupiny.

7.6 Backend API a synchronizácia vzťahov

Backend API má klasické CRUD endpointy pre hlavné modely: technologies, components, damage scenarios, controls, attack steps, threat scenarios, threat classes, control groups, cybersecurity goals a projekty. Okrem toho poskytuje endpointy pre výpočet rizík, risk treatment, report, MITRE dáta a LLM asistenta.

Jednotlivé endpointy používajú serializéry z `serializers.py`. Serializéry nerobia iba jednoduché mapovanie databázových polí. Obsahujú aj spätnú kompatibilitu pre staršie textové hodnoty. Napríklad impact hodnoty môžu byť zadane ako čísla, ale aj ako slová typu `low`, `medium`, `high`, `critical`. Podobne CIA bitmask vie spracovať hodnoty ako `C`, `I`, `A`, `CIA`, `111` alebo číselné hodnoty. Vďaka tomu je API tolerantnejšie a lepšie sa používa aj z LLM návrhov alebo z testovacích nástrojov.

V endpointoch je viacero synchronizačných helperov. Funkcia `sync_threat_scenarios` sa používa napríklad pri attack stepoch a controls. Ak sa vytvára objekt s názvom threat scenaria alebo so zoznamom threat scenarios, backend zabezpečí správne prepojenie. Ak má attack step komponent, komponent sa doplní aj do threat scenaria. Podobne `sync_threat_scenario_components` zjednocuje komponenty threat scenaria podľa priamych komponentov, komponentov z attack steps a komponentov z compromise záznamov.

Pri damage scenarios sa používa `sync_damage_scenario_concerns`. Tá kontroluje, či každý concern odkazuje na komponent, ktorý je povolený cez pripojené threat scenarios. Neplatné concerns sú odstránené. Následne sa z concerns vypočíta súhrnná CIA hodnota. Toto riešenie udržiava model konzistentný aj pri postupnej editácii z grafu, tabuľky alebo detailového panelu.

Frontend nad týmto API používa všeobecné funkcie `createModel`, `updateModel` a `deleteModel`. Tie podľa typu objektu vedú zavolať správny endpoint. Pri vytváraní objektov frontend pridáva `project_id` z aktuálneho projektu. Pri editácii sa zmeny ukladajú automaticky z detailového panelu s oneskorením 500 ms, aby sa pri písaní neposielala požiadavka po každom stlačení klávesy.

7.7 Grafický pohľad

Grafický pohľad je hlavný spôsob práce s modelom. Je implementovaný cez knižnicu `@xyflow/react`. Každý modelový objekt sa zobrazuje ako karta. Karta obsahuje názov, krátky popis, typ objektu a niekoľko metadát podľa typu. Napríklad komponent ukazuje počet komunikačných vzťahov a technológií, damage scenario ukazuje CIA a IL, attack step ukazuje komponent a počet threat scenarios, control ukazuje komponent a počet mitigovaných attack steps.

Graf sa skladá z uzlov a hrán. Uzly vznikajú zo všetkých modelových máp v lokálnom store. Z grafu sú momentálne vynechané cybersecurity goals, pretože tie sú lepšie čitateľné v tabuľkovom pohľade a detailoch. Hrany vznikajú z reálnych vzťahov medzi objektmi:

- komponent komunikuje s komponentom,
- komponent používa technológiu,
- data entity patrí ku komponentu alebo používa technológiu,
- control patrí ku komponentu,
- attack step patrí ku komponentu,
- attack step je prepojený s controls,
- attack step nadväzuje na predchádzajúci attack step,
- attack step alebo threat scenario majú threat class,

- threat scenario obsahuje komponenty, attack steps a damage scenarios,
- damage scenario je späť prepojené s threat scenarios,
- compromise odkazuje na komponent a threat scenario.

Používateľ môže karty presúvať po ploche. Pozície sa ukladajú do `localStorage` pod kľúčom odvodeným od projektu. V menu **File** je možné layout uložiť do JSON súboru a neskôr ho načítať. Po načítaní layoutu frontend nastaví uložené pozície na existujúce uzly.

Graf podporuje aj filtrovanie. Používateľ môže filtrovať podľa názvu a podľa typu objektu. Filtrovanie ovplyvňuje iba zobrazenie grafu, nie dáta v projekte. Pri kliknutí na kartu sa vybraný objekt nastaví ako `selectedItem` a otvorí sa v pravom detailovom paneli. Ak používateľ vyberie objekt v strome, graf sa naň vie automaticky centrovat.

Dôležitá funkcionálna je vytváranie spojení priamo v grafe. Každá karta má `connection handles`. Pri spojení dvoch kariet frontend zistí typ zdrojového a cieľového objektu a cez funkciu `isConnectable` overí, či medzi nimi existuje povolený vzťah. Ak áno, zavolá `addConnection`. Tá nájde správny smer vzťahu a správne pole. Napríklad pri spojení attack stepu s threat scenariom sa upraví zoznam `threat_scenarios` na attack stepe alebo zoznam `attack_steps` na threat scenarii podľa toho, ktorá strana vlastní daný vzťah v dátovom modeli.

Týmto spôsobom môže používateľ model skladať vizuálne. Nemusí si pamätať ID objektov ani manuálne editovať všetky zoznamy vzťahov. Aplikácia zároveň zabráňuje duplicitnému prepojeniu a nepovolí vzťahy, ktoré dátový model nepodporuje.

7.8 Strom projektu a vytváranie súvisiacich objektov

Ľavý panel v grafickom režime obsahuje strom projektu. Strom je načítaný z backendu cez funkciu `getProjectTree`. Zobrazuje komponenty a pod nimi súvisiace technológie, damage scenarios, attack steps, threat scenarios a ďalšie odkazy. Strom má vyhľadávanie, takže pri väčšom projekte sa dá rýchlo nájsť konkrétny scenár alebo komponent.

Strom funguje aj ako navigácia. Kliknutím na položku sa nastaví vybraný objekt. Detailový panel ho otvorí na editáciu a graf sa naň môže zamerať. Ak je vybraný objekt, v strome sa zobrazí tlačidlo na pridanie súvisiaceho modelu. Po kliknutí sa otvorí dialóg, v ktorom sa dá buď vybrať existujúci objekt a pripojiť ho, alebo vytvoriť nový objekt a hneď ho pripojiť k aktuálne vybranému.

Tento istý princíp je použitý aj v detailovom paneli. Vzťahy nie sú zobrazené iba ako text. Sú to klikateľné značky, ktoré umožňujú rýchlo prejsť na súvisiaci objekt. Vedľa zoznamu vzťahov je tlačidlo na pridanie ďalšieho vzťahu. Používateľ teda môže začať napríklad na komponente, vytvoriť k nemu attack step, z attack stepu vytvoriť threat scenario, k threat scenarii pridať damage scenario a potom vrátiť sa k attack stepu a doplniť controls.

7.9 Detailový panel a formuláre

Pravý panel je detailový editor. Jeho obsah závisí od typu vybraného modelu. Pre jednoduché objekty, ako technológia, komponent alebo data entity, obsahuje hlavne názov a popis. Pre TARA objekty obsahuje špecifické polia.

Pri damage scenarii sa zobrazuje názov, popis, affected components, CIA checkboxy a impact hodnoty. CIA sa zadáva cez checkboxy pre Confidentiality, Integrity a Availability. Ak damage scenario má pripojené threat scenarios, formulár vie odvodiť komponenty, ktoré môžu byť affected. Používateľ potom nastavuje CIA na úrovni konkrétnych komponentov. Formulár zároveň ukazuje vypočítaný IL.

Pri attack stepe sa zadáva názov, popis, required access, MITRE technika, threat class a všetky attack feasibility faktory. Formulár obsahuje aj niekoľko praktických príkladov, napríklad CAN frame injection, diagnostic service abuse, firmware extraction alebo credential/key compromise. Výber príkladu predvyplní názov, popis, required access a faktorové hodnoty. Používateľ si ich vie upraviť podľa konkrétneho projektu.

Pri controle formulár obsahuje názov, popis, výber control class, pripojené threat scenarios a faktorové hodnoty. Ak používateľ vyberie control class, frontend preberie jej defaultné hodnoty. Pod hodnotami sa zobrazuje aj vypočítaná attack feasibility controlu. Pri attack stepe sa navyše zobrazuje efektívne AFL s aktívnymi controls, ak je vybraná control skupina a daný step má relevantné controls.

Pri threat scenarii sa zadáva názov, popis, threat class a involved components. Aj tu sú dostupné príklady, napríklad spoofed in-vehicle network messages, unauthorized diagnostic access, malicious software update alebo remote service exploitation. Pri cybersecurity goal sa nastavuje názov, popis, CAL a väzby na damage scenarios a controls.

Detailový panel teda nie je len prezeranie dát. Je to hlavný editor, ktorý umožňuje priebežne ukladať zmeny, mazať objekty, pridávať vzťahy a prechádzať medzi súvisiacimi časťami modelu.

7.10 Tabuľkový pohľad

Tabuľkový pohľad je určený na kontrolu a analýzu. Vľavo má navigáciu medzi tabuľkami a pri každej tabuľke zobrazuje počet položiek. Implementované tabuľky sú:

- Threat Scenarios,
- Damage Scenarios,
- Attack Steps,
- Controls,
- Risks,
- Cybersecurity Goals.

Každá tabuľka má hlavné stĺpce, ktoré dávajú prehľad o danom type objektu. Riadky sú klikateľné. Kliknutím sa jednak nastaví vybraný objekt v detailovom paneli a jednak sa riadok rozbalí. Rozbalená časť obsahuje detailnejšie informácie, napríklad popis, súvisiace komponenty, zoznam attack steps, impact hodnoty alebo attack feasibility faktory.

Tabuľka damage scenarios ukazuje ID, názov, IL, CIA a threat scenarios. Po rozbalení ukazuje popis, impact detail a concerns. Tabuľka attack steps ukazuje komponent, threat scenarios a AFL. Ak je aktívna control skupina, tabuľka vie zobrazovať efektívne AFL s controls a pôvodné základné AFL. Podobne tabuľka risks vie zobrazovať efektívne RL, ak sa po aplikovaní controls zmení.

Tabuľka risks je najdôležitejšia pre výslednú analýzu. Backend endpoint `/api/risk/` vytvorí riadky na základe kombinácií threat scenarios a damage scenarios. Pre každý riadok vypočíta AFL, IL a RL. Ak damage scenario obsahuje concerns, vzniká riadok aj na úrovni konkrétneho affected komponentu. Ak ide o starší damage scenario bez concerns, endpoint vie stále vytvoriť legacy riadok zo súhrnnej CIA hodnoty.

V rozbalenom detaile risku je aj editor risk treatment. Používateľ môže vybrať rozhodnutie Avoid, Reduce, Share alebo Accept a dopísať rationale. Uloženie zavolá endpoint `/api/risk_treatment/`. Po uložení sa riziká znovu načítajú, aby tabuľka zobrazovala aktuálny treatment.

Tabuľkový pohľad má tlačidlo **New** pre tie tabuľky, kde dáva zmysel vytvárať nové objekty. Tým sa otvorí rovnaký formulárový dialóg ako v grafickom pohľade. Používanie aplikácie teda nie je viazané na jeden workflow. Používateľ môže modelovať vizuálne alebo tabuľkovo podľa toho, čo je v danej chvíli prehľadnejšie.

7.11 Control skupiny a porovnávanie mitigácií

Control skupiny sú dostupné z toolbaru cez tlačidlo **Controls**. Dialóg zobrazuje dve vstavané skupiny **All Controls** a **No Controls**, a potom všetky používateľské skupiny z databázy. Používateľ môže skupinu vybrať, vytvoriť novú, upraviť existujúcu alebo ju zmazať.

Pri vytváraní alebo úprave skupiny sa zadáva názov, popis a zoznam controls. Výber aktívnej skupiny ovplyvňuje výpočty zobrazované vo frontende. Ak je aktívna skupina **No Controls**, attack steps a risks ukazujú základný stav. Ak je aktívna skupina **All Controls**, aplikujú sa všetky controls, ktoré sú pripojené k attack steps. Pri vlastnej skupine sa aplikujú iba vybrané controls.

Technicky sa aktívna skupina ukladá vo **Zustand** store ako hodnota `activeControlGroupId`. Môže to byť `all`, `none` alebo ID reálnej skupiny. Funkcia `getActiveControlIds` podľa toho vráti zoznam aktívnych control ID. Tento zoznam potom používajú výpočty v tabuľkách a formulároch.

Na backende sa control groups ukladajú v modeli **ControlGroup**. Report generátor ich používa na samostatnú tabuľku „Control Scenarios per Risk“. Tam sa pre každý risk ukáže základný risk level a risk level pri jednotlivých control skupinách. To je užitočné najmä pri porovnávaní alternatívnych mitigácií. Používateľ vie napríklad ukázať, že minimálna sada opatrení zníži RL z 4 na 3, ale plná sada opatrení zníži RL z 4 na 2.

7.12 Asset Identification assistant

V režime **Assistants** je implementovaný asistent **Asset Identification**. Jeho cieľom je pomôcť používateľovi vytvoriť základné damage a threat scenarios pre existujúce assets. Asistent prechádza komponenty a data entities, ktoré sú už načítané v stave projektu. Pre každý asset vytvorí tri návrhy – Confidentiality, Integrity a Availability.

Návrhy sa zobrazujú v tabuľkovom zozname. Pri každom návrhu je stav: Proposed, Accepted, Rejected alebo Creating. Používateľ môže návrh prijať, odmietnuť, obnoviť odmietnutý návrh alebo použiť **Apply All**. Pri jednom asete sa dá použiť aj **Apply All** iba pre jeho tri CIA časti.

Keď používateľ prijme návrh, frontend vytvorí nové threat scenario a nové damage scenario. Threat scenario má názov typu „Confidentiality compromise of ...“ a description obsahuje interný source tag. Damage scenario má názov typu „Loss of Confidentiality for ...“. Ak asset patrí ku komponentu, damage scenario dostane concern na daný komponent s príslušnou CIA hodnotou. Threat scenario je pripojené ku komponentu a damage scenario je pripojené k threat scenariu.

Source tag v popise nie je viditeľný prvok pre koncového používateľa, ale používa sa na rozpoznanie, či už bol návrh vytvorený. Asistent vie nájsť damage scenario, ktoré obsahuje rovnaký source tag, a potom ho zobrazí ako „Created“. Zároveň vie nájsť aj existujúce scenáre, ktoré už pokrývajú rovnaký komponent a CIA časť, a vtedy namiesto „Accept“ zobrazí možnosť „Add another“. Tým sa znižuje riziko, že asistent bude vytvárať duplicitné scenáre bez upozornenia.

Tento asistent je jednoduchší ako plný LLM asistent v konzole, ale má výhodu, že je deterministický. Nepotrebuje externé API a funguje vždy nad aktuálnym modelom. Je vhodný hlavne v začiatku analýzy, keď má projekt definované komponenty a dáta, ale ešte chýbajú základné damage scenarios.

7.13 Konzola a LLM návrhy

Spodný panel aplikácie obsahuje konzolu. Konzola pôvodne mala slúžiť ako história akcií, ale vo finálnej implementácii je zároveň komunikačným rozhraním s TARA asistentom. Používateľ môže do nej napísať otázku, požiadať o analýzu aktuálneho modelu alebo požiadať o vytvorenie návrhov.

Frontend najprv odošle prompt na backend endpoint `/api/langchains/prepare/`. Spolu s promptom posiela históriu posledných správ a pamäť predchádzajúcich návrhov. Backend zostaví kontext projektu, ktorý obsahuje počty objektov, komponenty, technológie, damage scenarios, attack steps, threat scenarios, controls a mapu vzťahov. Tento kontext dá LLM modelu spolu so systémovým promptom.

Backend LLM časť je implementovaná v súboroch `langchains.py` a `langchain_tools.py`. Používa `langchain-deepseek` a nástroje definované cez LangChain tools. Model môže zavolať napríklad:

- `propose_technology`,
- `propose_component`,
- `propose_damage_scenario`,
- `propose_attack_step`,
- `propose_threat_scenario`,
- `propose_control`,
- `analyze_current_state`.

Dôležité je, že LLM priamo nezapíše do databázy. Nástroje iba pripravujú návrhy. Každý návrh má `tempId`, typ, názov, popis, rationale, payload a prípadné references na iné návrhy. Frontend tieto návrhy zobrazí v dialógu `LangchainProposalDialog`. Používateľ ich môže rozbaľiť, skontrolovať a až potom jednotlivo prijať alebo odmietnuť.

Pri prijatí návrhu frontend zavolá `createLangchainProposal`. Tá podľa typu návrhu vytvorí príslušný objekt cez štandardné model API. Ak návrhy medzi sebou odkazujú cez dočasné references, frontend ich vie po vytvorení zosúladiť cez `reconcileAcceptedProposals`. To umožňuje napríklad situáciu, kde LLM v jednej odpovedi navrhne nový attack step, nové damage scenario a threat scenario, ktoré ich má prepojiť. Kým objekty ešte nemajú databázové ID, vzťahy sa držia cez temp IDs. Po prijatí sa nahradia reálnymi ID.

Konzola má aj lokálny fallback. Ak backend assistant nie je dostupný alebo nie je nakonfigurovaný, frontend vie spraviť jednoduchú lokálnu analýzu stavu alebo pripraviť lokálne draft návrhy. Pri analýze spočíta objekty a upozorní na medzery, napríklad komponent bez damage scenarios, komponent bez attack steps alebo komponent bez controls. Táto funkcionality je užitočná pri vývoji a demonštrácii, pretože aplikácia ostáva použiteľná aj bez externého LLM providera.

LLM asistent si tiež pamätá stav návrhov. Ak používateľ návrh prijal, odmietol alebo jeho uloženie zlyhalo, táto informácia sa posiela späť do backendu ako `proposalMemory`. Systémový prompt explicitne hovorí modelu, aby neopakoval už prijaté, odmietnuté alebo čakajúce návrhy, pokiaľ používateľ nepožiadá o ich revíziu. Tým sa zlepšuje použiteľnosť pri konverzačnej práci typu „analyzuj model“, „navrhni chýbajúce controls“, „pokračuj“.

7.14 MITRE integrácia

Aplikácia obsahuje jednoduchú integráciu MITRE ATT&CK. Backend má súbor `mitre.py`, ktorý načítava STIX bundle a poskytuje endpointy pre tactics a techniques. Frontend má funkcie

`fetchMitreTactics` a `fetchMitreTechniques`, ktoré tieto dáta používajú vo formulároch.

V dátovom modeli sú MITRE väzby uložené priamo ako textové polia. Threat class má `mitre_tactic_id` a `mitre_tactic_name`. Attack step má `mitre_technique_id` a `mitre_technique_name`. Takéto riešenie je jednoduché a praktické. Aplikácia nemusí ukladať celý MITRE dataset do vlastnej databázy, ale zvolená taktika alebo technika ostane uložená pri konkrétnom TARA objekte.

MITRE teda vo výslednej aplikácii neslúži ako automatický generátor analýzy. Slúži skôr ako referenčná vrstva a slovník, ktorý pomáha pomenovať attack steps a threat classes konzistentne s existujúcou bezpečnostnou terminológiou.

7.15 Generovanie PDF reportu

Toolbar obsahuje tlačidlo **Generate Report**. Po kliknutí frontend zavolá endpoint `/api/report/s project_id`. Backend overí prístup k projektu a zavolá funkciu `generate_report_pdf`. Výsledkom je PDF súbor, ktorý frontend stiahne ako `tara-report-<projectId>.pdf`.

Report je generovaný pomocou knižnice `reportlab`. Graf rizík sa generuje cez `matplotlib`. Report obsahuje viacero sekcií:

- základné informácie o projekte,
- risk distribution chart,
- assets – data entities,
- assets – components,
- damage scenarios overview,
- cross-reference medzi damage a threat scenarios,
- threat scenarios and attack paths,
- attack steps table,
- controls table,
- risks table,
- control scenarios per risk,
- risk treatment table,
- cybersecurity goals.

Report používa rovnaké výpočty ako backend API. Threat scenarios sa prechádzajú spolu s attack steps a damage scenarios. Pre každú unikátnu dvojicu threat scenario + damage scenario sa vypočíta AFL, IL a RL. Potom sa tieto riadky používajú v risk tabuľke a v grafe rozloženia rizík.

Risk distribution chart zobrazuje maticu impact vs. attack feasibility. Farebné bunky zodpovedajú risk levelom a kruhy ukazujú počet rizík v jednotlivých bunkách. Tým report poskytuje rýchly vizuálny prehľad, či sa riziká sústreďujú v nízkych alebo vysokých oblastiach.

Sekcia control scenarios per risk porovnáva základný stav bez controls a všetky definované control groups. Pre každú control group backend znovu vypočíta efektívnu attack feasibility threat scenaria a z nej nový risk level. V PDF je tak vidno, ktoré opatrenia majú reálny vplyv na výsledné riziko.

Report je navrhnutý ako praktický výstup z aplikácie. Nemá nahrádzať všetku dokumentáciu TARA procesu, ale poskytuje export aktuálneho stavu projektu vo forme, ktorú je možné poslať, archivovať alebo použiť ako podklad pre ďalšiu diskusiu.

7.16 Vzťah medzi grafom, tabuľkami, detailmi a backendom

Jedna z najdôležitejších častí výslednej implementácie je, že aplikácia nemá tri oddelené editory. Graf, tabuľky a detailový panel sú tri rôzne pohľady na rovnaký model. Z technického pohľadu to zabezpečuje `model-store.ts`.

Pri otvorení projektu store zavolá `getProjectState` a `getProjectRisks`. Projektový stav sa uloží ako mapy podľa typu modelu. Riziká sa ukladajú ako pole `risks`. Komponenty, scenáre a controls sa potom používajú na tvorbu grafových uzlov, tabuľkových riadkov, formulárov aj zoznamov vzťahov.

Keď používateľ upraví objekt v detailovom paneli, najprv sa zmení lokálny stav cez `setItem`. Po krátkom oneskorení sa zavolá `saveItem`, ktorý pošle update na backend. Ak ide o typ, ktorého zmena môže ovplyvniť vzťahy a riziká, napríklad threat scenario, damage scenario alebo attack step, frontend po uložení znovu načíta projektový stav. Tým sa aktualizujú odvodené polia, concerns, riziká aj strom.

Keď používateľ vytvorí nový objekt pre typ podporovaný API, `addItem` ho uloží cez backend. Ak API vráti objekt s projektom, frontend znovu načíta celý projektový stav. Je to jednoduchšie a bezpečnejšie ako ručne skladať všetky odvodené vzťahy na klientovi. Pri mazaní objektu sa volá `deleteItem`, ktorý pri podporovaných typoch zmaže objekt cez API a potom ho odstráni z lokálneho store.

Vzťahy sa spravujú cez `addConnection` a `deleteConnection`. Tieto funkcie najprv určia, ktorá strana vzťahu vlastní príslušné pole. Pri typoch s plnou API podporou sa zmena odošle do backendu a následne sa obnoví stav projektu. Niektoré pomocné typy, najmä data entities a compromises, sú v aktuálnej implementácii skôr načítavané a zobrazované ako samostatne spravované. Pri nich teda priame zmeny vo frontende nemajú rovnakú perzistentnú CRUD podporu ako komponenty, attack steps, threat scenarios, damage scenarios, controls alebo cybersecurity goals.

7.17 Príklad práce s aplikáciou

Typický workflow môže vyzeráť takto:

1. Používateľ sa prihlási a otvorí projekt „Auto security“.
2. V graph view vytvorí komponenty ako Infotainment Head Unit, Central Gateway, TCU alebo Engine ECU.
3. Ku komponentom pripojí technológie ako CAN, Bluetooth/BLE, Cellular alebo Automotive Ethernet.
4. V asistentovi Asset Identification pre komponenty, prípadne existujúce data entities, prijme návrhy CIA strát. Tým sa vytvoria prvé threat scenarios a damage scenarios.
5. V detailoch damage scenaria nastaví konkrétne affected components a impact hodnoty pre safety, financial, operational a privacy.
6. Vytvorí attack steps, napríklad Gateway Firewall Bypass, Unauthorized Firmware Flashing alebo CAN Command Injection. Pri každom doplní faktorové hodnoty pre attack feasibility.

7. Attack steps pripojí k threat scenarios a threat scenarios k damage scenarios. V grafe tým vznikne stopa od útoku cez hrozbu až po škodu.
8. Vytvorí controls, napríklad pravidlá firewallu, secure boot, autentizáciu diagnostiky alebo integritnú kontrolu konfigurácie, a pripojí ich k relevantným attack steps.
9. V table view otvorí Risks a skontroluje AFL, IL a RL. Prepína control skupiny a sleduje, ako sa mení efektívne riziko.
10. Pre vybrané riziká doplní risk treatment a rationale.
11. Vytvorí cybersecurity goals, ktoré odvodí z damage scenarios a prepojí ich s controls.
12. Nakoniec vygeneruje PDF report.

Tento tok nie je v aplikácii vynútený. Používateľ môže začať grafom, tabuľkami, asistentom alebo konzolou. Dôležité je, že všetky cesty zapisujú do rovnakého dátového modelu a výsledné riziká sa počítajú z rovnakých vzťahov.

7.18 Obmedzenia a možné rozšírenia

Výsledná implementácia pokrýva hlavné časti TARA workflow, ale stále existujú oblasti, ktoré by sa dali rozšíriť. Niektoré časti sú implementované pragmaticky, aby aplikácia bola použiteľná v rámci tímového projektu.

Hierarchia komponentov z pôvodného návrhu nie je vo finálnom modeli riešená ako parent-child strom. Komponenty majú komunikačné vzťahy a strom projektu sa skladá z odvodených vzťahov, ale nie je tam explicitný rodičovský komponent. Ak by sa aplikácia ďalej rozvíjala, dalo by sa pridať pole `parent` na komponent a tým podporiť komplexnejšie systémy.

Data entities sú v aplikácii podporené len čiastočne. Existujú v databázovom modeli a frontend ich vie zobraziť, ale aktuálne nemajú samostatný plnohodnotný CRUD workflow cez API a UI. Výpočet rizík je vedený hlavne cez komponenty, threat scenarios a damage scenarios. Pre detailnejšiu asset-centric analýzu by sa dali doplniť samostatné endpointy a damage scenario concerns rozšíriť aj na data entities, nielen komponenty.

LLM asistent pripravuje návrhy a vie analyzovať aktuálny stav, ale konečné rozhodnutie ostáva na používateľovi. To je správne z hľadiska bezpečnosti aj kvality TARA analýzy. V budúcnosti by sa však dali pridať špecializované nástroje pre návrh attack paths z MITRE dát, kontrolu konzistencie podľa ISO/SAE 21434 alebo automatické generovanie cybersecurity goals z rizík.

MITRE integrácia je zatiaľ referenčná. Aplikácia vie použiť tactics a techniques, ale nevykonáva hlboké mapovanie medzi automotive komponentmi a MITRE technikami. Toto by mohlo byť ďalším smerom rozvoja, hlavne ak by sa pridali aj CAPEC alebo CWE dáta, ktoré boli popísané v skorších kapitolách.

Report je funkčný a pokrýva hlavné výstupy, ale dá sa ďalej zlepšiť po vizuálnej a metodickej stránke. Napríklad by mohol obsahovať samostatné kapitoly pre metodiku hodnotenia, históriu zmien alebo explicitné odkazy na ustanovenia ISO/SAE 21434.

7.19 Zhrnutie výsledku

Finálna aplikácia poskytuje ucelené prostredie na tvorbu TARA analýzy. Používateľ v nej vie spravovať projekty, definovať systémové komponenty a technológie, vytvárať damage scenarios, threat scenarios a attack steps, dopĺňať controls, počítat riziká, porovnávať control skupiny, zapisovať risk treatment, definovať cybersecurity goals a exportovať report.

Najväčšia praktická hodnota implementácie je v prepojení týchto častí. TARA model nie je uložený ako izolované tabuľky. Threat scenarios prepájajú attack steps s damage scenarios, damage scenario concerns držia CIA dopad na konkrétnych komponentoch, controls sa aplikujú na attack steps a control groups ukazujú ich vplyv na výsledné riziká. Grafický pohľad pomáha tieto vzťahy pochopiť, tabuľkový pohľad ich pomáha skontrolovať a detailový panel ich umožňuje upravovať.

Backend pritom zabezpečuje konzistenciu a výpočty. Frontend poskytuje rôzne ergonomické spôsoby práce s tým istým modelom. Asistenti a LLM konzola pridávajú podporu pri tvorbe obsahu, ale vždy tak, že používateľ má nad výsledkom kontrolu. Výsledkom je aplikácia, ktorá dokáže podporiť celý základný tok TARA analýzy od modelovania systému až po reportovateľný výsledok.